



Deep Learning

Lecture 4.1: Convolutional Neural Networks



Today's Learning Objectives

Be able to answer:

- What are convolutional layers?
 - What is convolution and what bias does it provide?
 - What is padding? Stride?
 - How are convolutional layers related to linear layers?
- What are pooling layers?
 - Why do we use them?
- What are convolutional neural networks?
 - What is a receptive field?





What is deep learning? Matching inductive bias to problem structure.



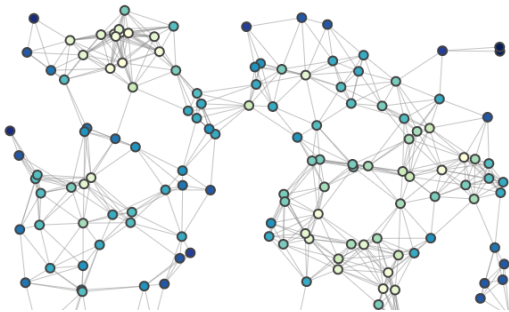
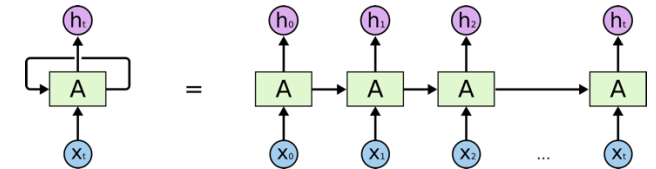
Translation Invariance
with Convolution



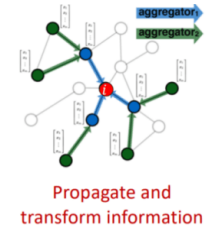
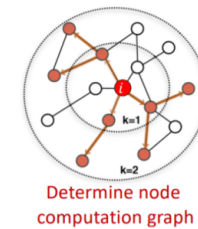
This burrito place
is yummy and fun!



Recurrence & Long-
Range Dependencies

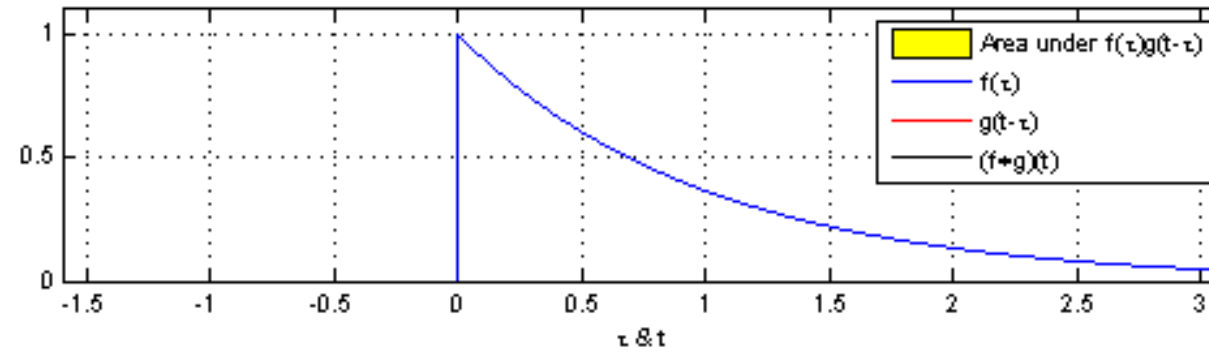
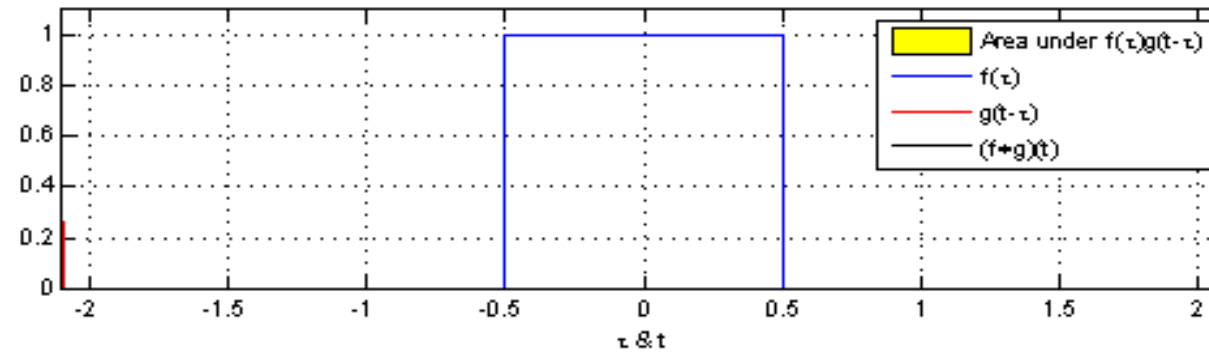


Message Passing



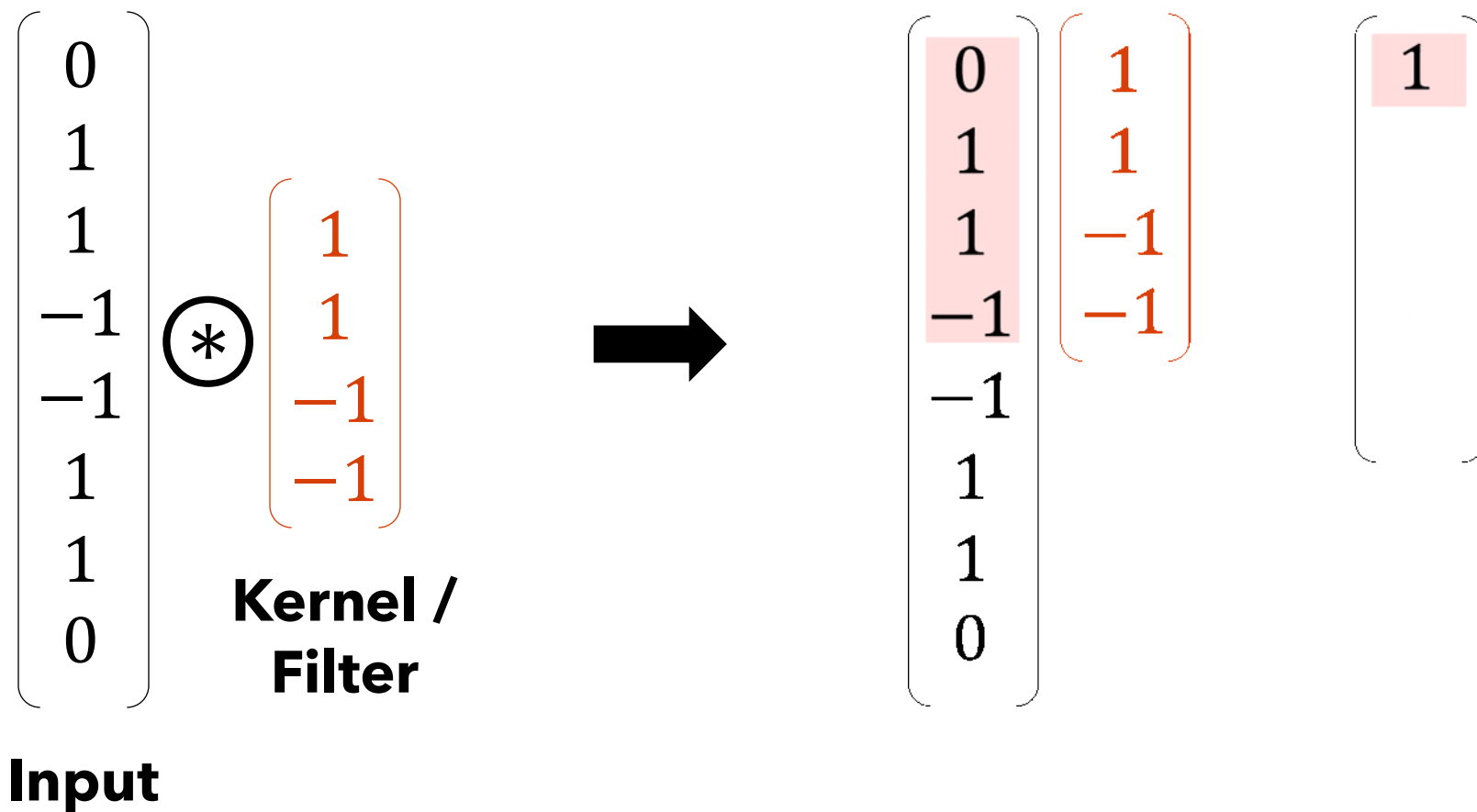


$$(f \circledast g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$





$$(f \circledast g)(t) = \sum_{\tau=0}^N f[t + \tau]g[\tau], \quad g \in \mathbb{R}^N$$





Goal: Classify any signal with two 1's followed by two -1's as positive.

$$a = \max(\text{sign}(\mathbf{W}^T \mathbf{x} + \mathbf{b})) \longrightarrow a = \max(\text{sign}(\mathbf{w} \circledast \mathbf{x} + \mathbf{b}))$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 1 \\ 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$\mathbf{W}$

**SAVED PARAMETERS BY MAKING A LOCALITY ASSUMPTION!
ALSO KNOWN AS "TRANSLATIONAL INVARIANCE"**

Goal: Have a feature respond to the word "Alexa"

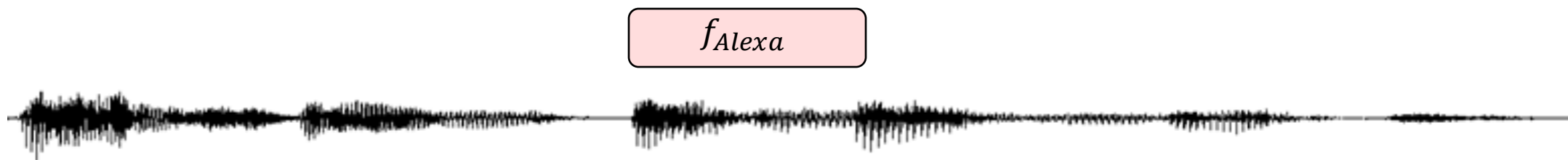
f_{Alexa}



Goal: Have a feature respond to the word "Alexa"



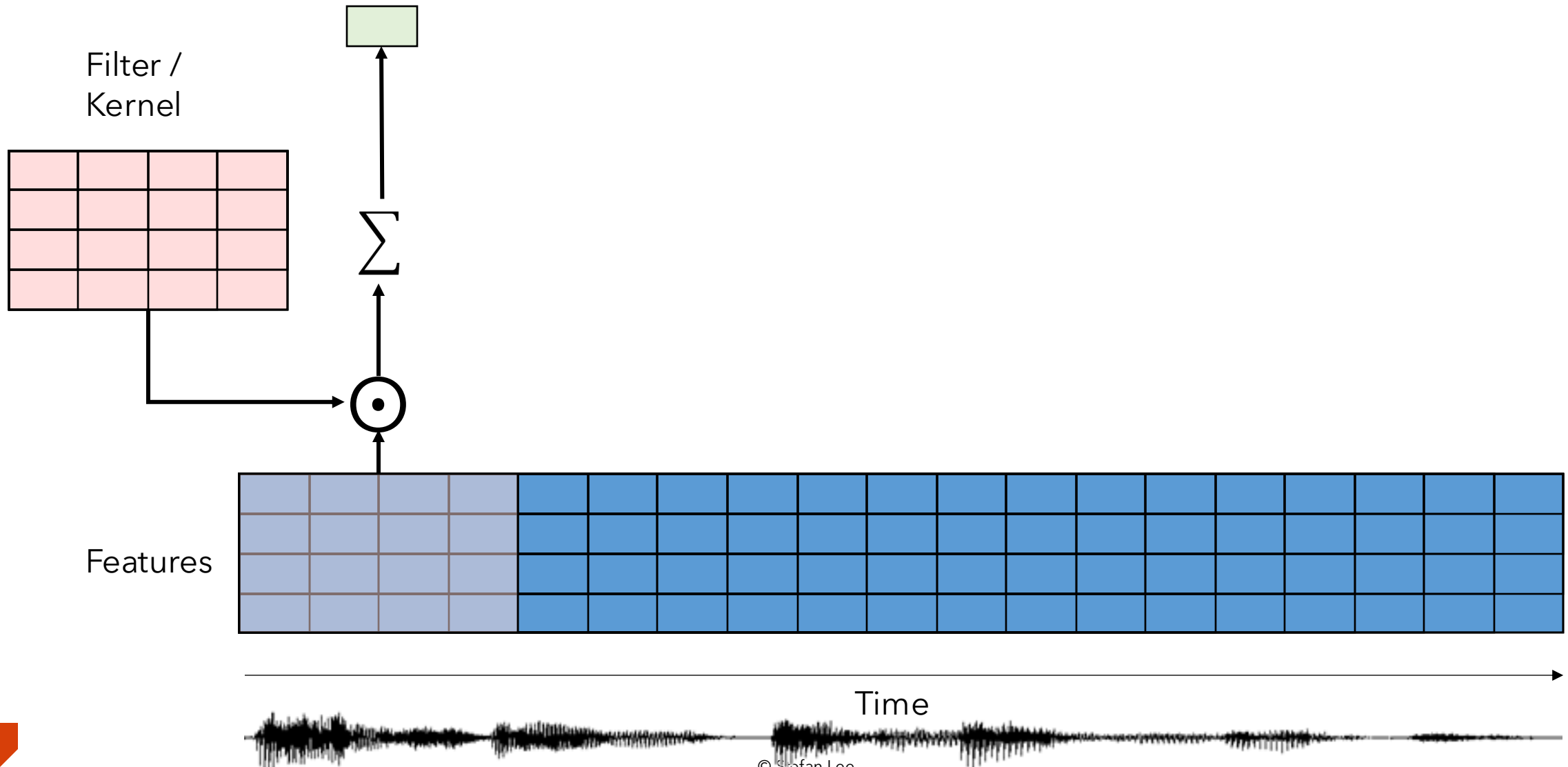
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

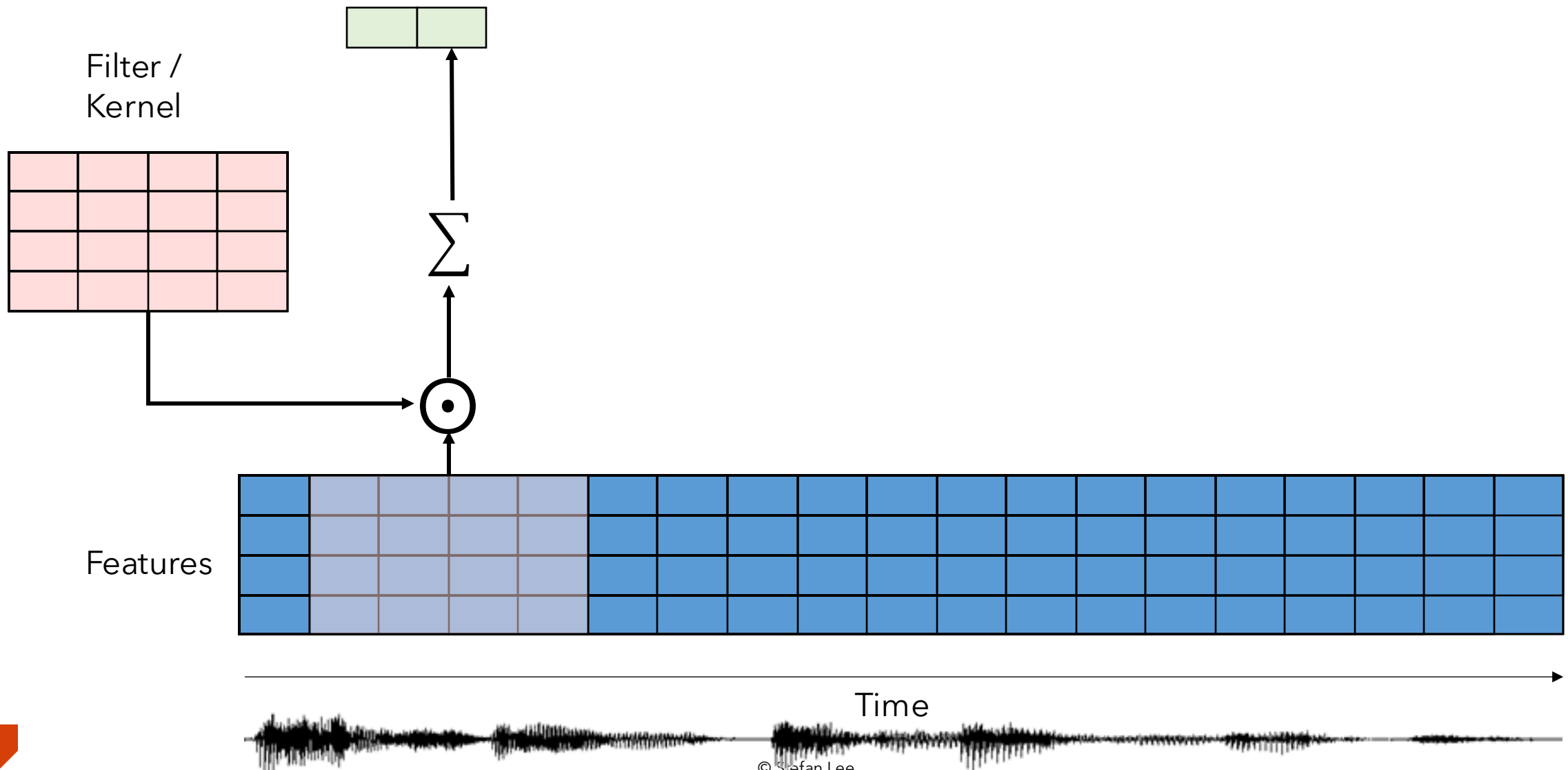
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

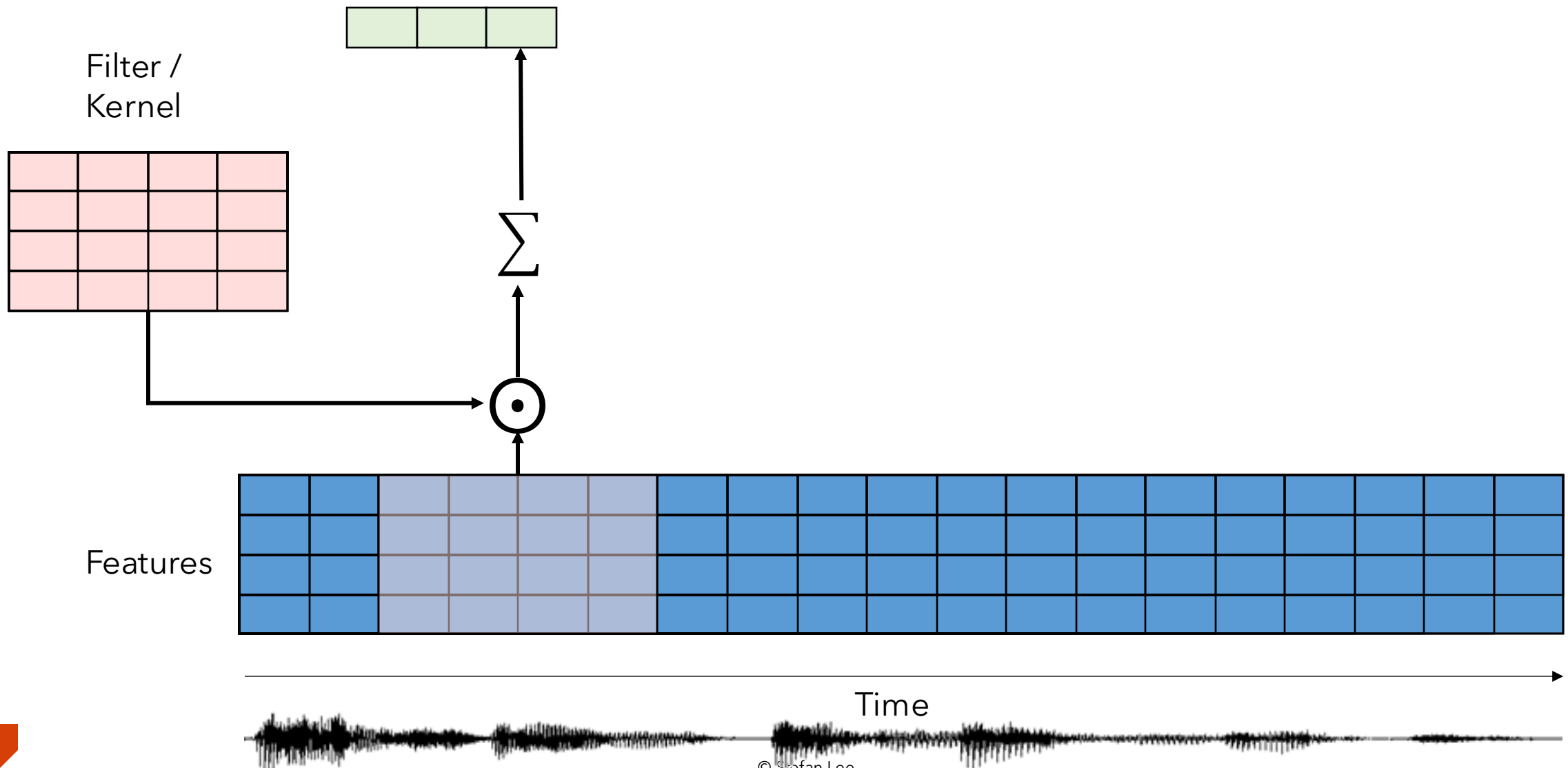
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

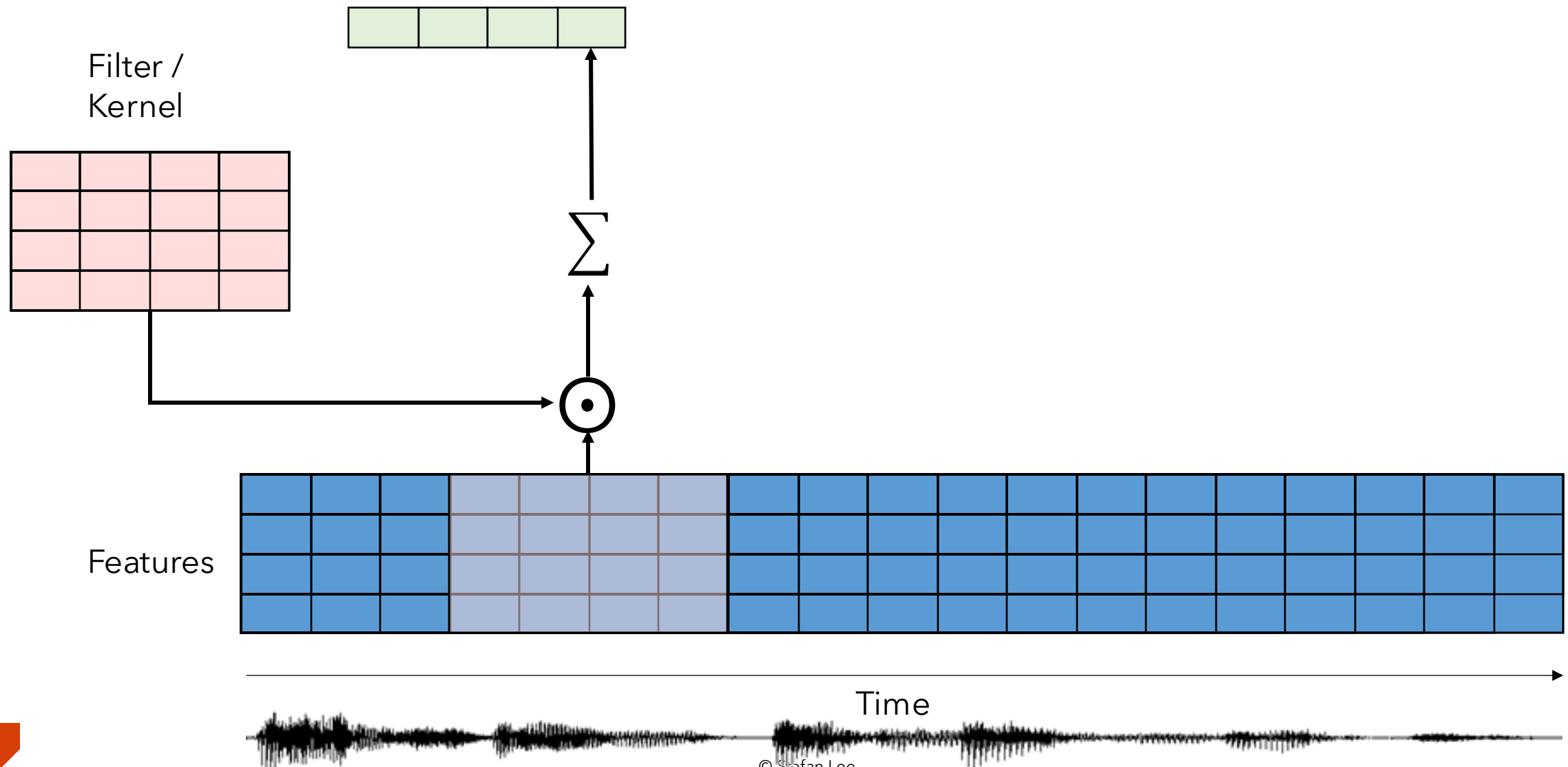
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

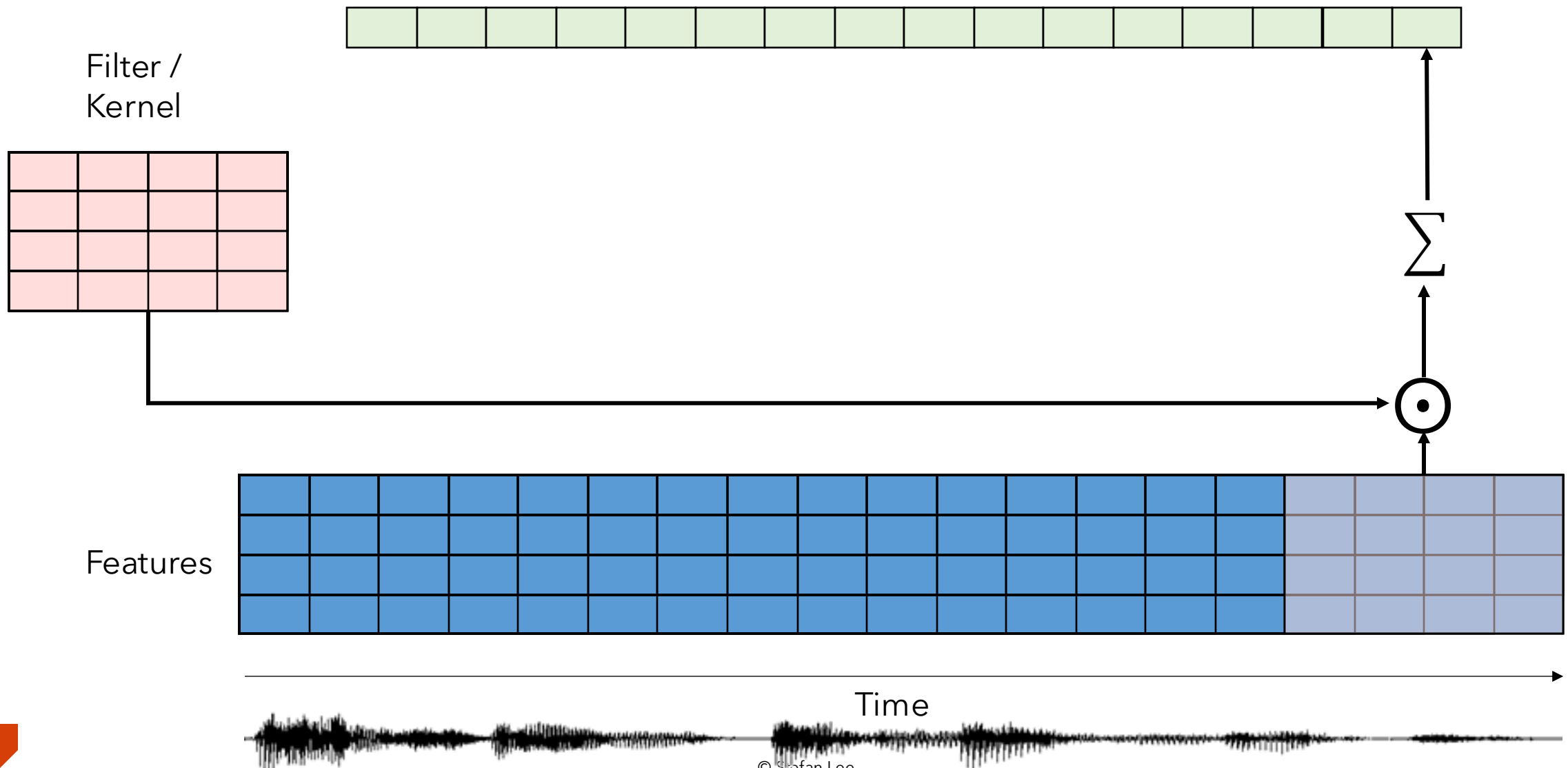
Goal: Have a feature respond to the word "Alexa"



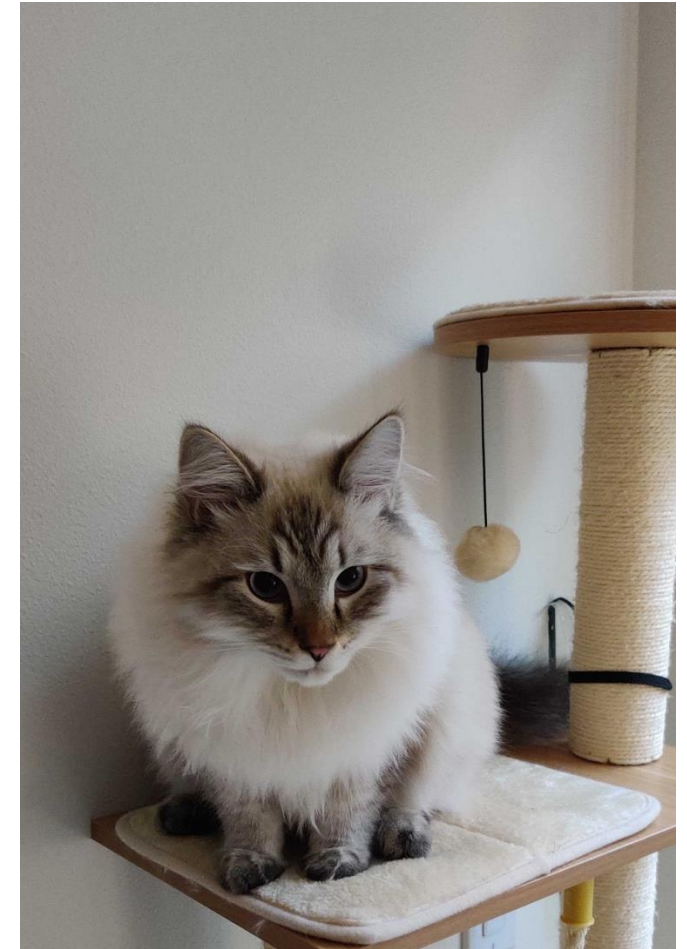
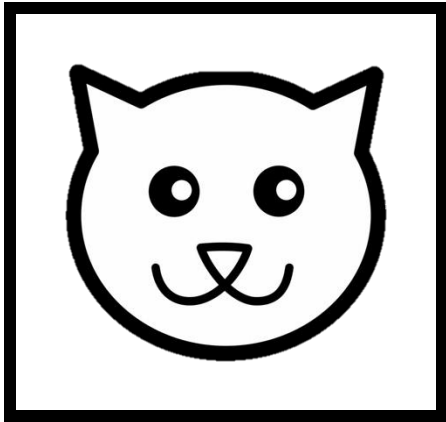


Example where the Locality Bias Fits (1D)

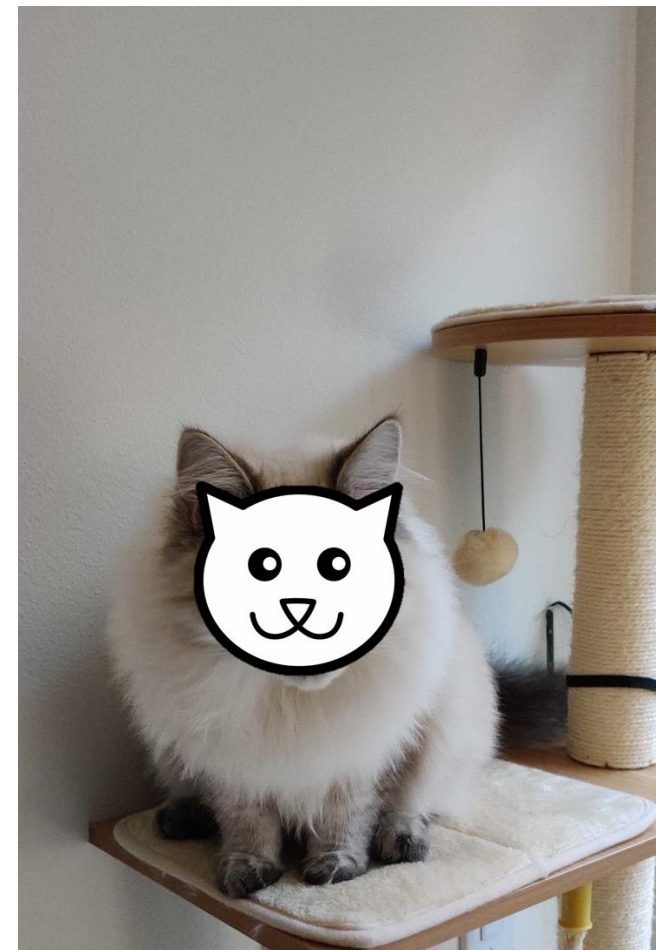
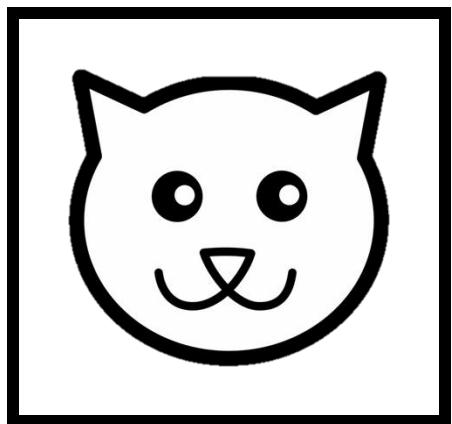
Goal: Have a feature respond to the word "Alexa"



Goal: Have a feature respond to cat faces

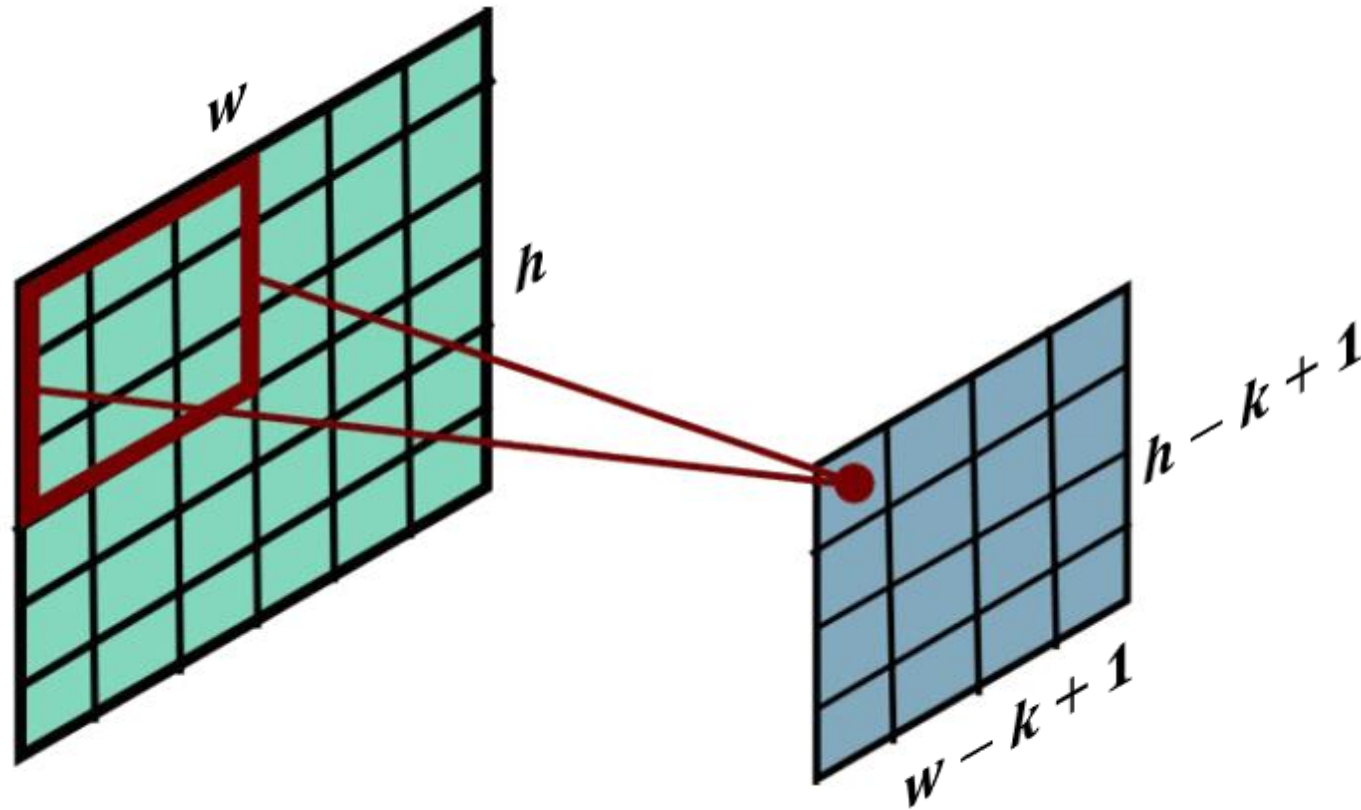


Goal: Have a feature respond to cat faces



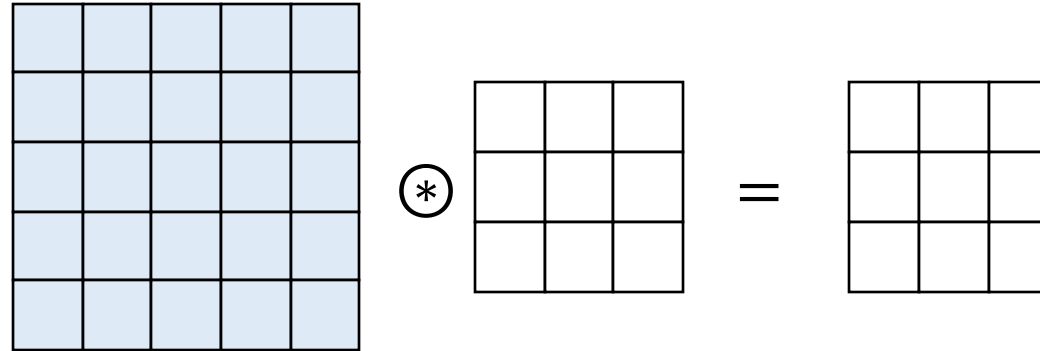


$$(f \circledast g)(m, n) = \sum_{i=-k/2}^{k/2} \sum_{j=-k/2}^{k/2} f[m+i, n+j] g\left[i + \frac{k}{2}, j + \frac{k}{2}\right], \quad g \in \mathbb{R}^{k \times k}$$

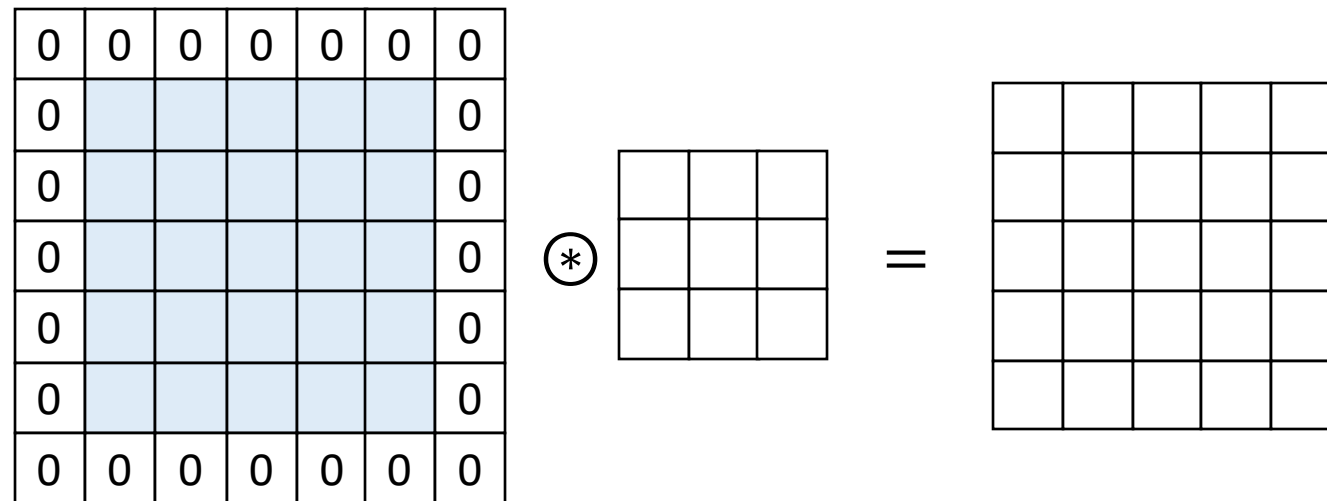




No Padding - Spatial dimensions reduce.



Padding=1 - Output dim same as input (for this filter size).





0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

$\otimes$

$=$

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0						0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

$\otimes$

$=$



Padding in general with padding of p :

$$\begin{matrix} w + 2p \\ \hline \begin{matrix} h + 2p \\ \hline \text{blue square} \end{matrix} \end{matrix} \circledast \begin{matrix} k \\ \hline \text{kernel} \end{matrix} = \begin{matrix} w + 2p - k + 1 \\ \hline \text{output} \end{matrix}$$

If $p = \frac{k-1}{2}$, no change in spatial dimension.

- To make this an integer, we often use odd filter size k



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

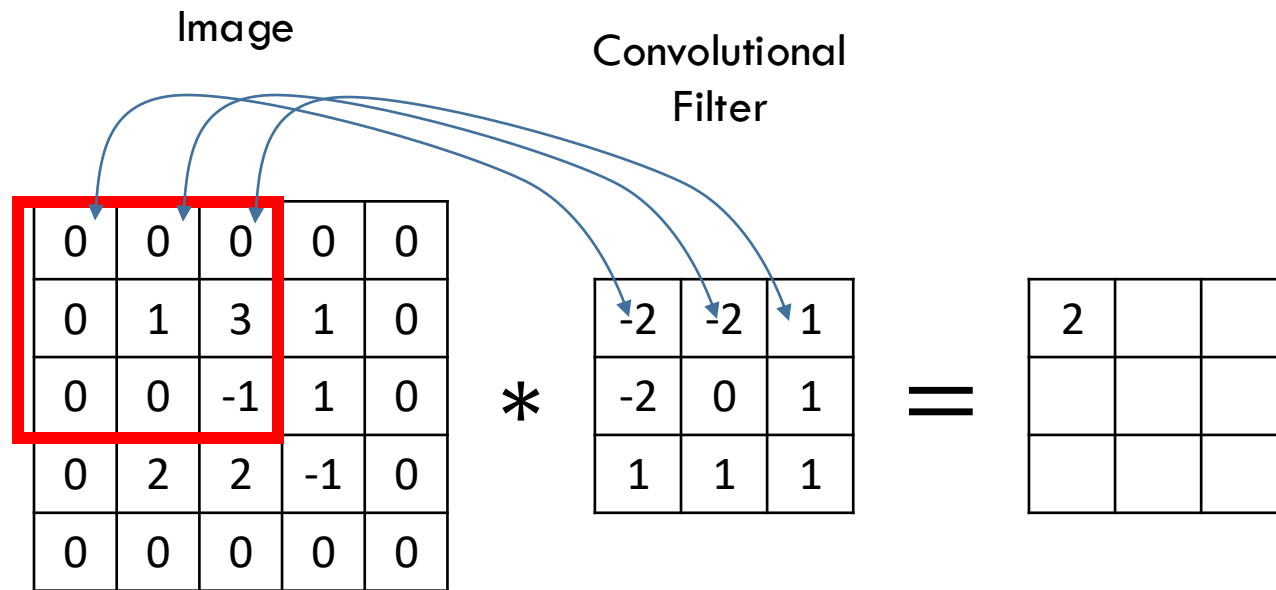
 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$



2D Convolution with Padding



$$3 \times 1 + (-1) \times 1 = 2$$

$$[W * X]_{ij} = \sum_{\Delta_i, \Delta_j} W_{\Delta_i \Delta_j} X(i + \Delta_i, j + \Delta_j)$$

$$\text{e.g. } \Delta_i \in \{-1, 0, 1\}, \Delta_j \in \{-1, 0, 1\}$$



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$

2	-1	

$$1 \times (-2) + 1 \times 1 + 1 \times (-1) + 1 \times 1 = -1$$



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 *

-2	-2	1
-2	0	1
1	1	1

 =

2	-1	-6

What if:

0	0	3	3	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 *

-2	-2	1
-2	0	1
1	1	1

 =

2	-1	-18



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$

2	-1	-6
4		



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$

2	-1	-6
4	-3	



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$

2	-1	-6
4	-3	-5



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$

2	-1	-6
4	-3	-5
1		



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$

2	-1	-6
4	-3	-5
1	-2	



2D Convolution with Padding

0	0	0	0	0
0	1	3	1	0
0	0	-1	1	0
0	2	2	-1	0
0	0	0	0	0

 $*$

-2	-2	1
-2	0	1
1	1	1

 $=$

2	-1	-6
4	-3	-5
1	-2	-2



Early Days of 2D Convolution: Edge Detection with Fixed Filter

Sobel edge detection filter

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

H11	H12	H13
H21	H22	H23
H31	H32	H33

*

F11	F12	F13	F14	F15	F16
F21	F22	F23	F24	F25	F26
F31	F32	F33	F34	F35	F36
F41	F42	F43	F44	F45	F46
F51	F52	F53	F54	F55	F56
F61	F62	F63	F64	F65	F66

=

G11	G12	G13	G14	G15	G16
G21	G22	G23	G24	G25	G26
G31	G32	G33	G34	G35	G36
G41	G42	G43	G44	G45	G46
G51	G52	G53	G54	G55	G56
G61	G62	G63	G64	G65	G66

I



Convolution



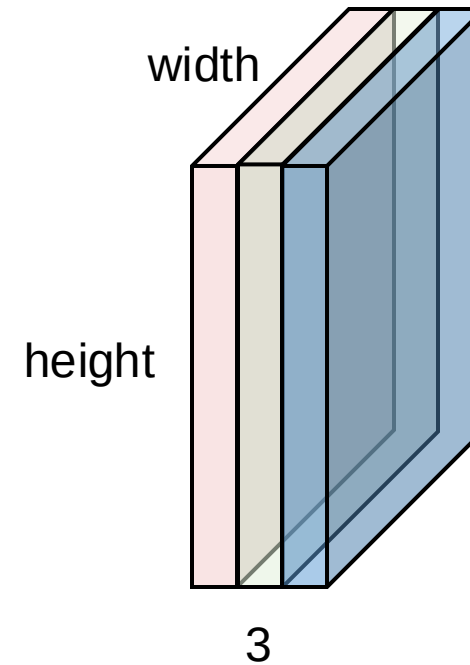
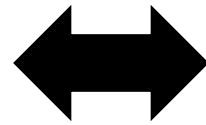
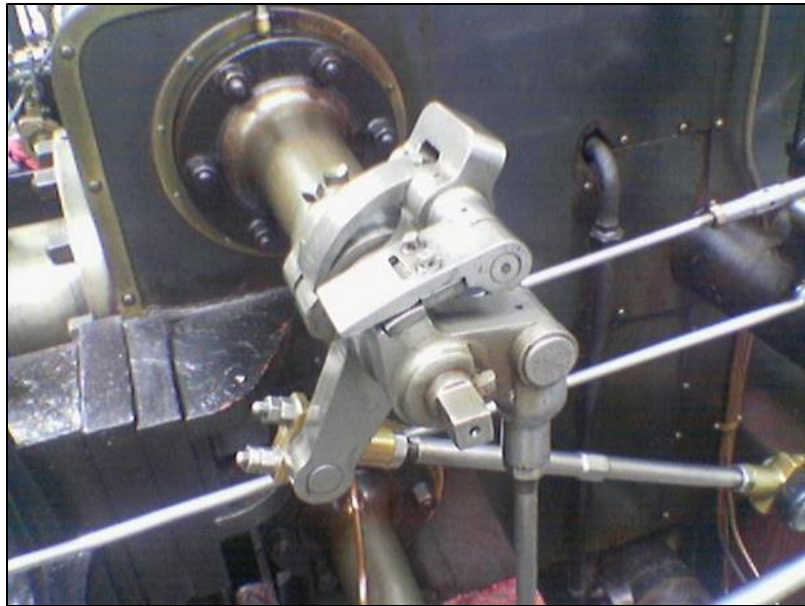
$$I * Gx + I * Gy$$





Discrete Convolution in Across Channels

Color images are $w \times h \times 3$ matrices



How to perform 2D convolution on a 3D volume?

Discrete Convolution Across Channels

$5 \times 5 \times 3$

6	2	3	4	5
2	5	7	2	0
0	3	5	6	4
9	8	2	0	7
6	2	2	3	0

$\otimes$

$3 \times 3 \times 3$

2	0	-1
0	1	0
1	0	-1

9

+

3×3

-26		

$\otimes$

2	2	3	1	5
2	7	7	2	0
0	3	3	4	4
9	8	2	0	6
6	6	2	3	3

-17 =

+

$\otimes$

1	2	3	4	3
1	5	5	2	1
0	3	5	4	4
3	2	7	0	2
4	2	6	3	0

1	0	-1
-1	0	-2
1	0	-1

-18

Discrete Convolution Across Channels

$5 \times 5 \times 3$

6	2	3	4	5
2	5	7	2	0
0	3	5	6	4
9	8	2	0	7
6	2	2	3	0

$\otimes$

$3 \times 3 \times 3$

2	0	-1
0	1	0
1	0	-1

9

+

3×3

-26		

$\otimes$

1	2	-1
2	0	-2
1	-2	-1

-17 =

+

$\otimes$

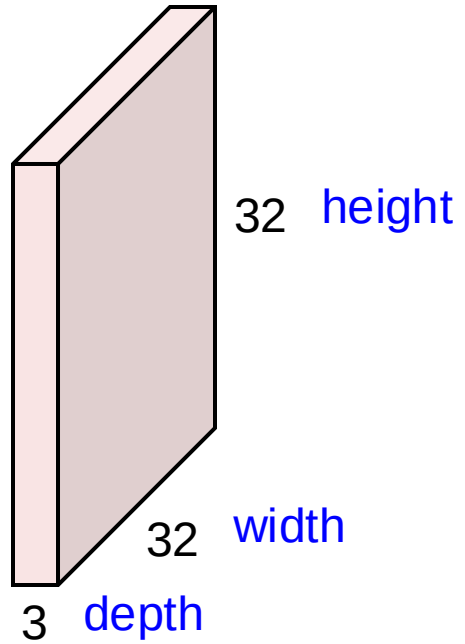
1	0	-1
-1	0	-2
1	0	-1

-18

1	2	3	4	3
1	5	5	2	1
0	3	5	4	4
3	2	7	0	2
4	2	6	3	0

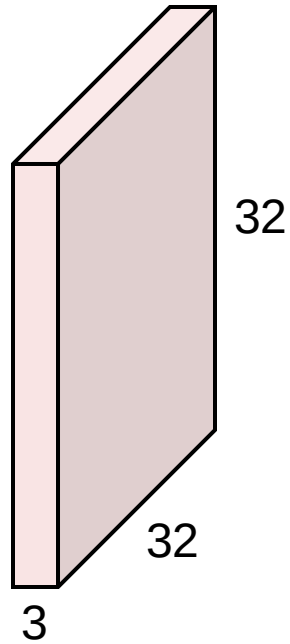


32x32x3 image -> preserve spatial structure





32x32x3 image



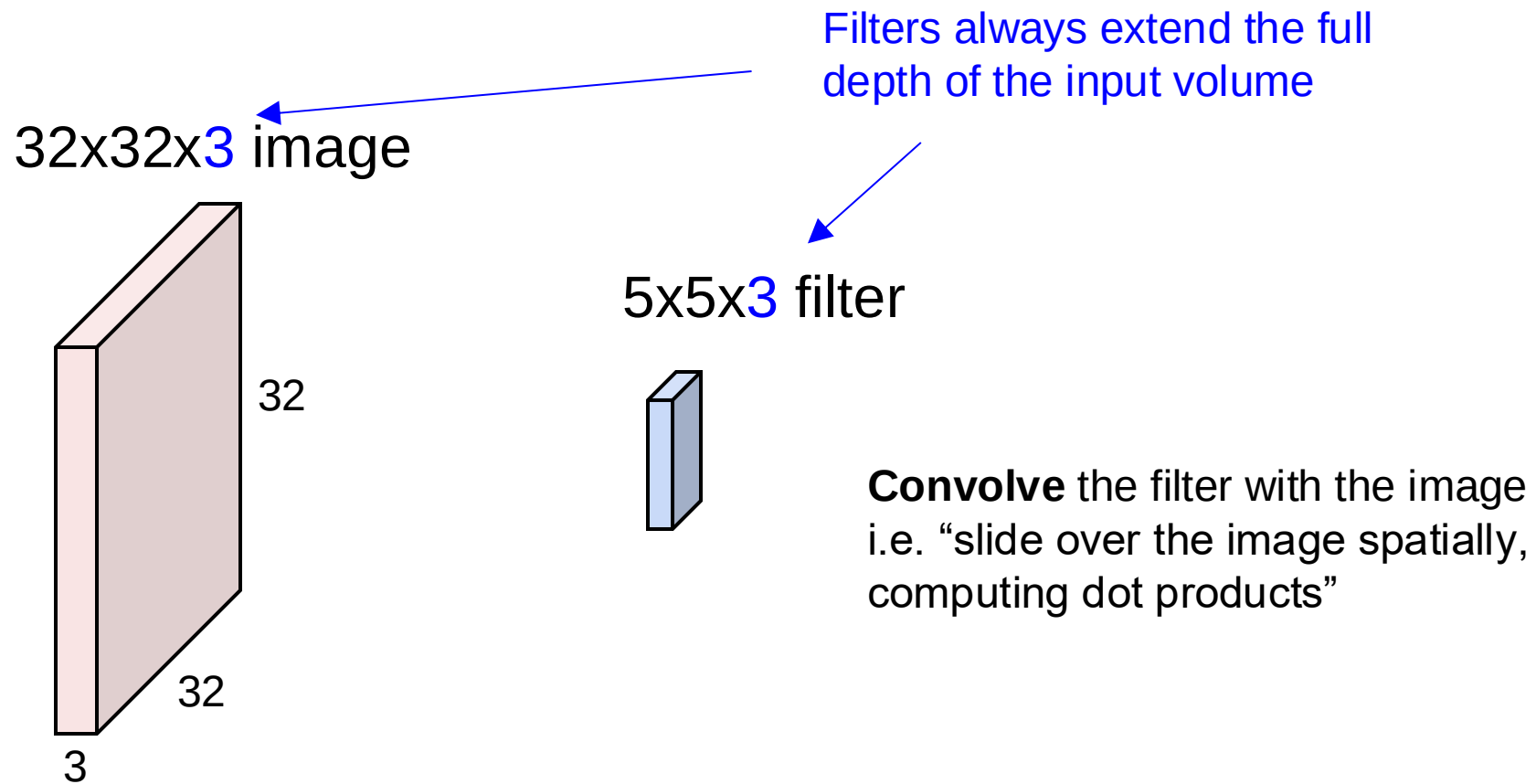
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

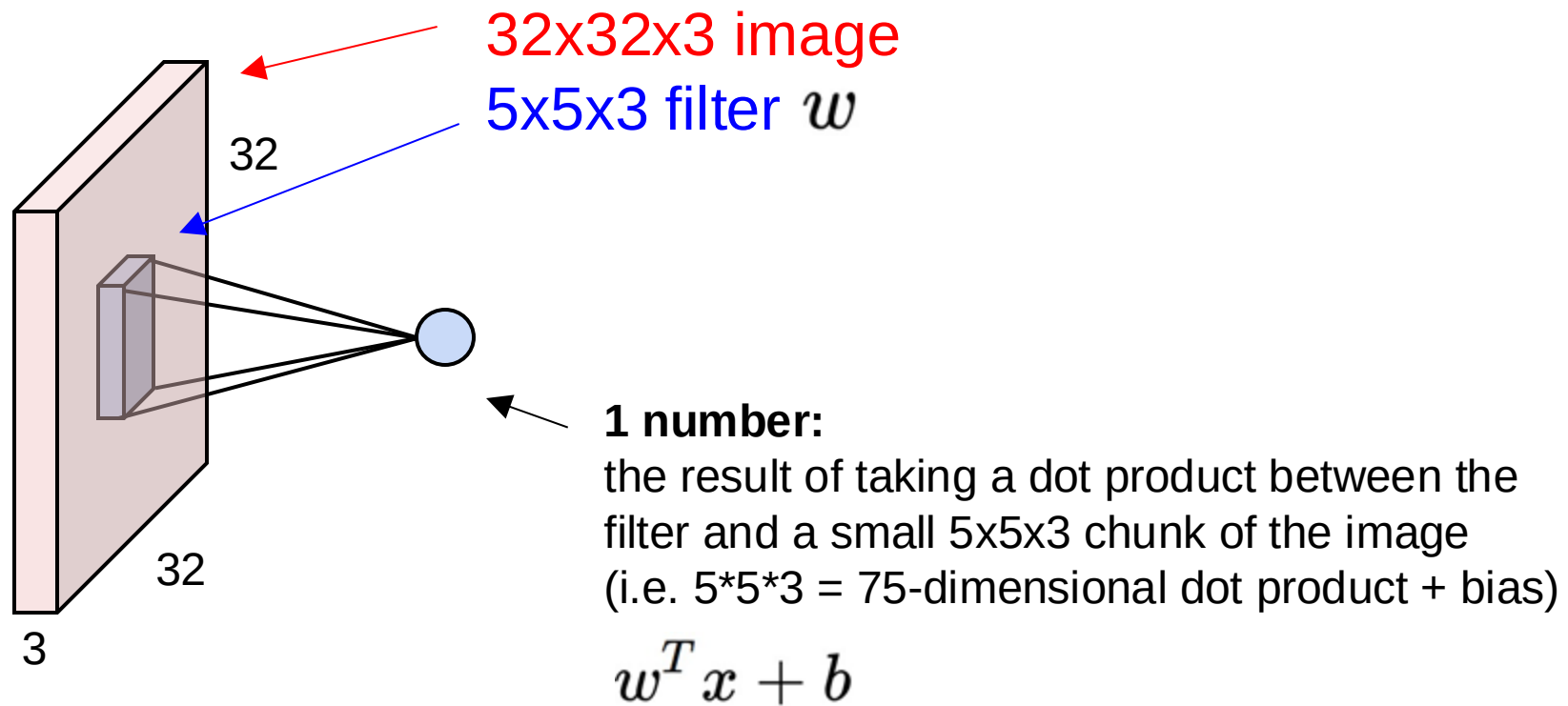


Discrete Convolution in 2D



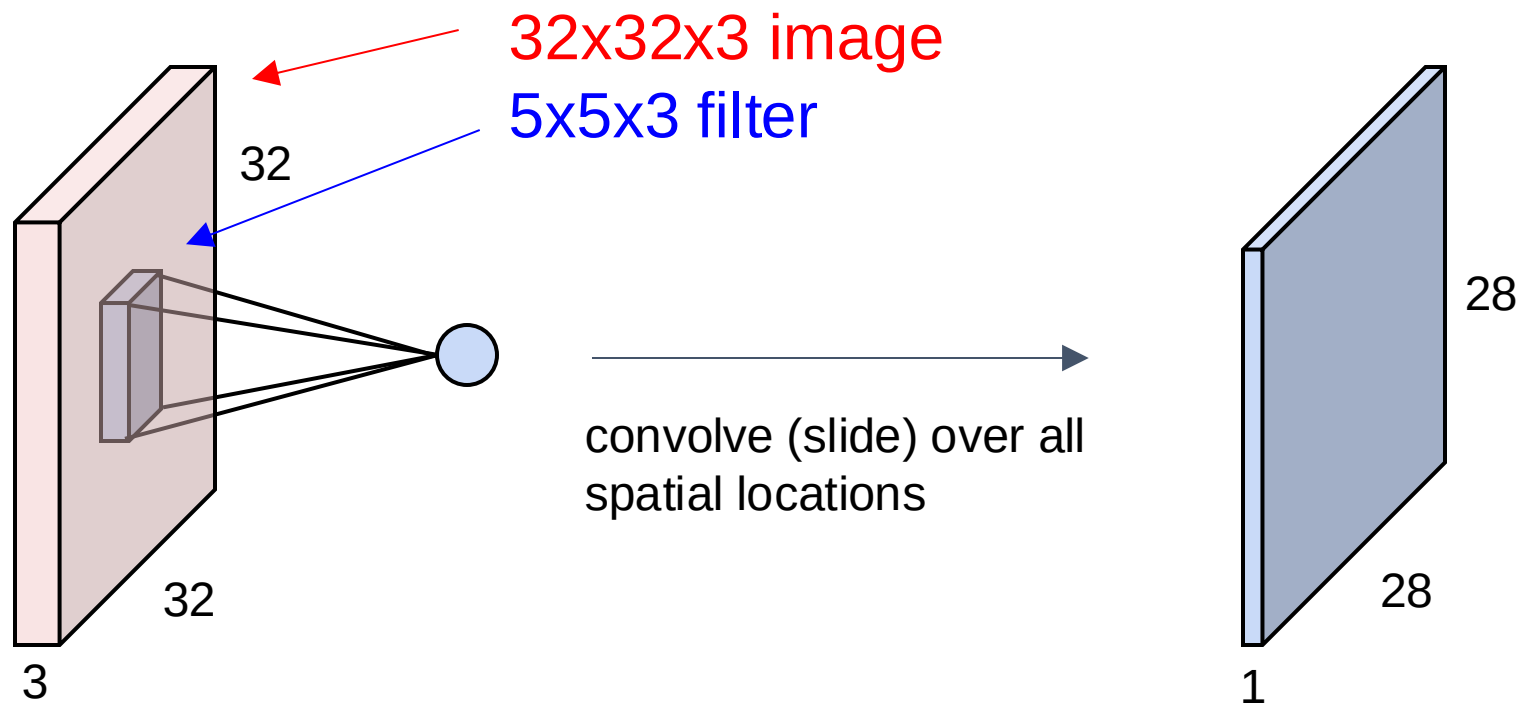


Discrete Convolution in 2D





Discrete Convolution in 2D



(No zero padding involved here,
hence, size of the image reduced)



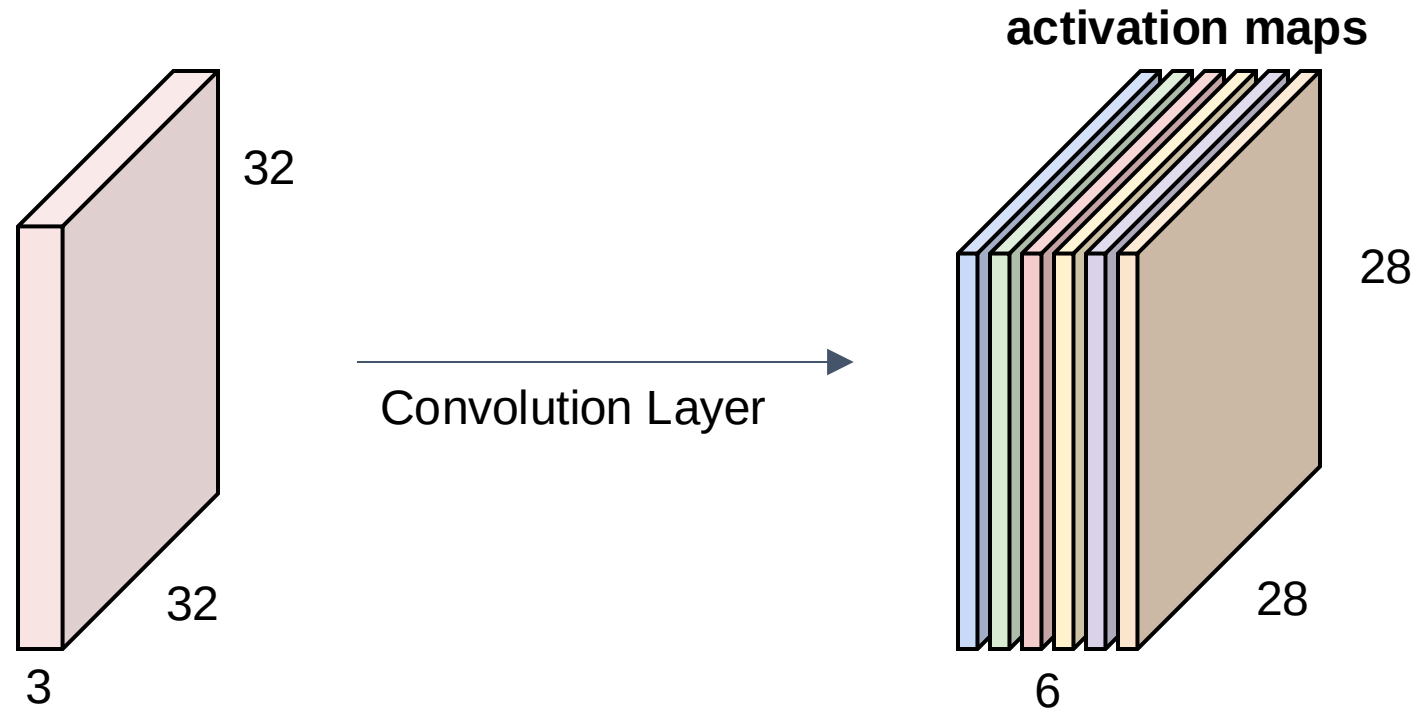
consider a second, **green** filter



(No zero padding involved here,
hence, size of the image reduced)



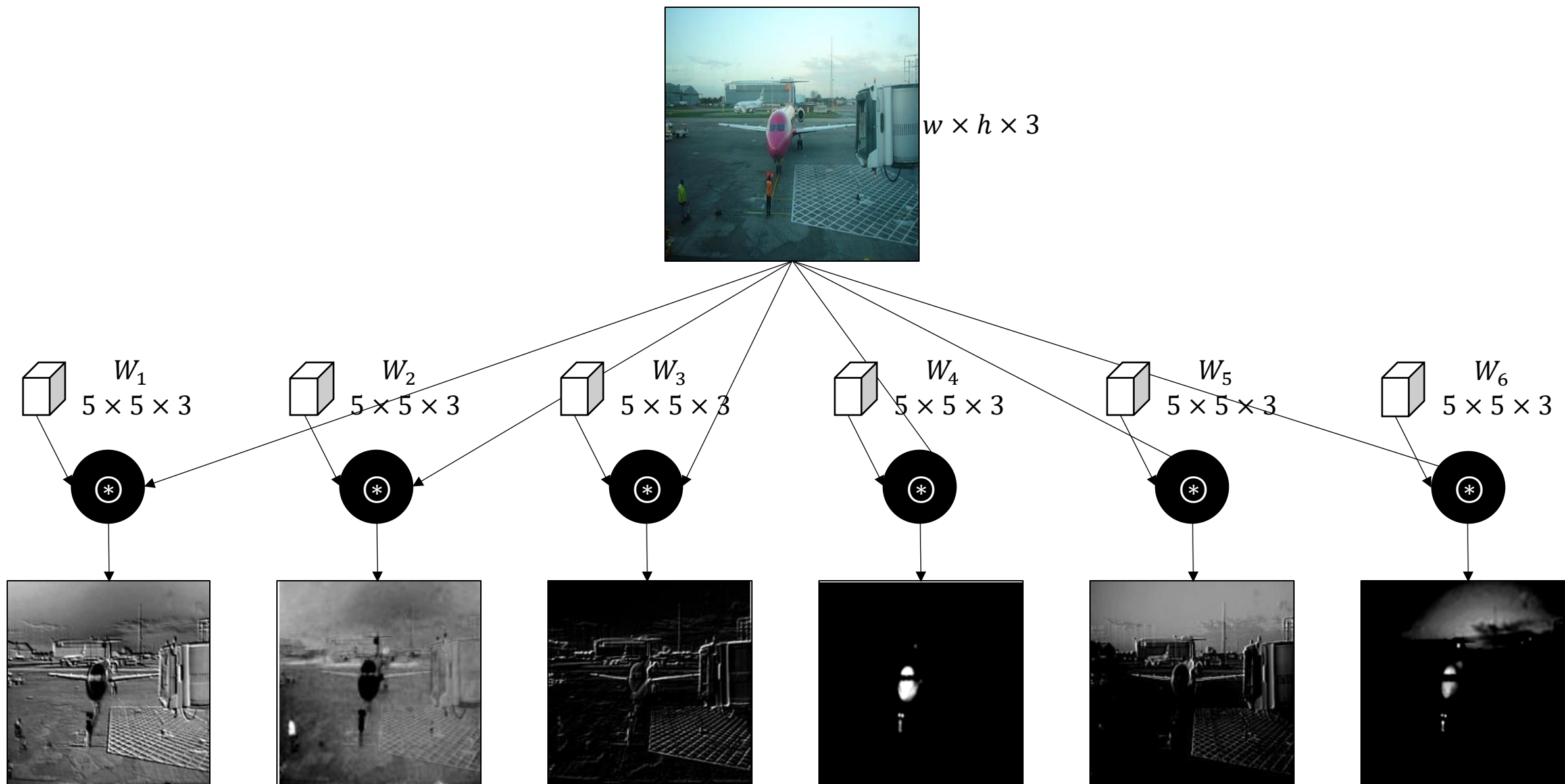
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a “new image” of size 28x28x6!

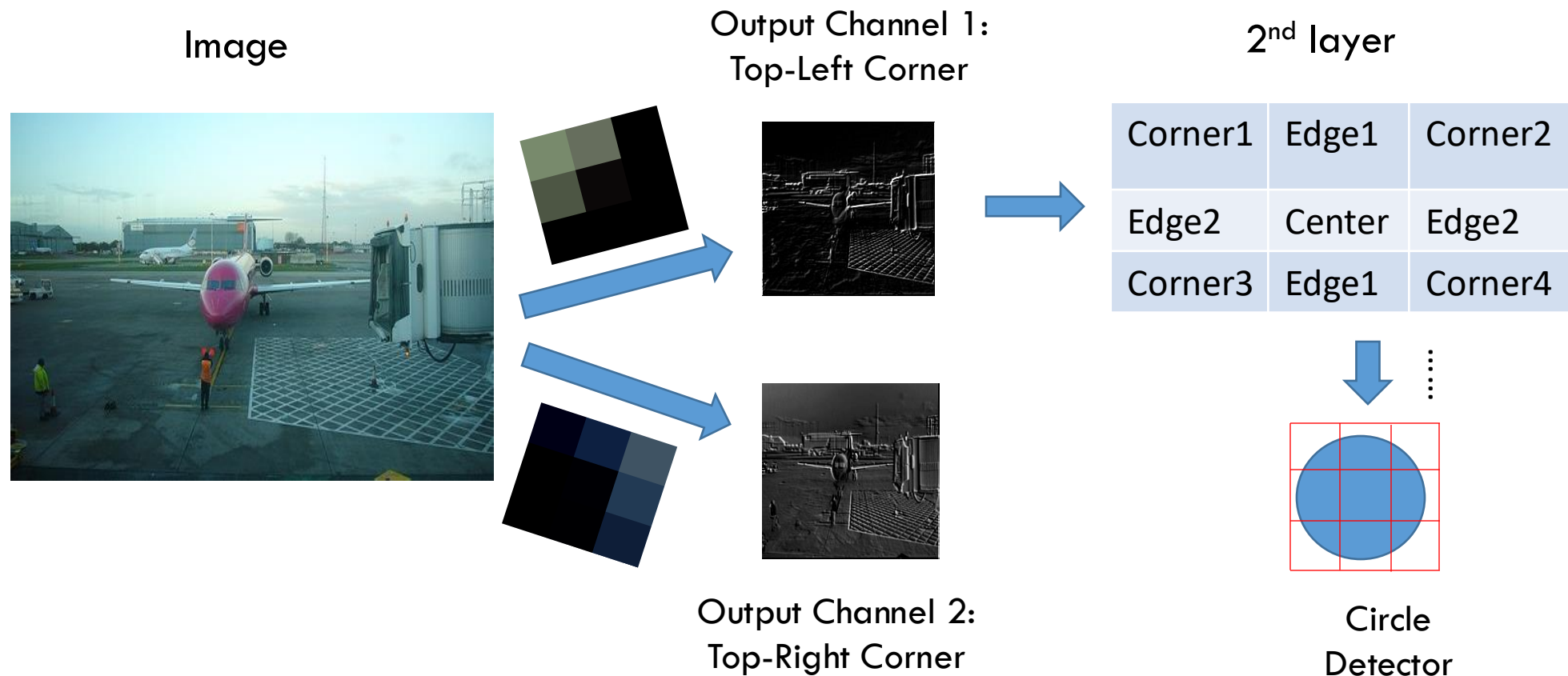


Discrete Convolution in 2D



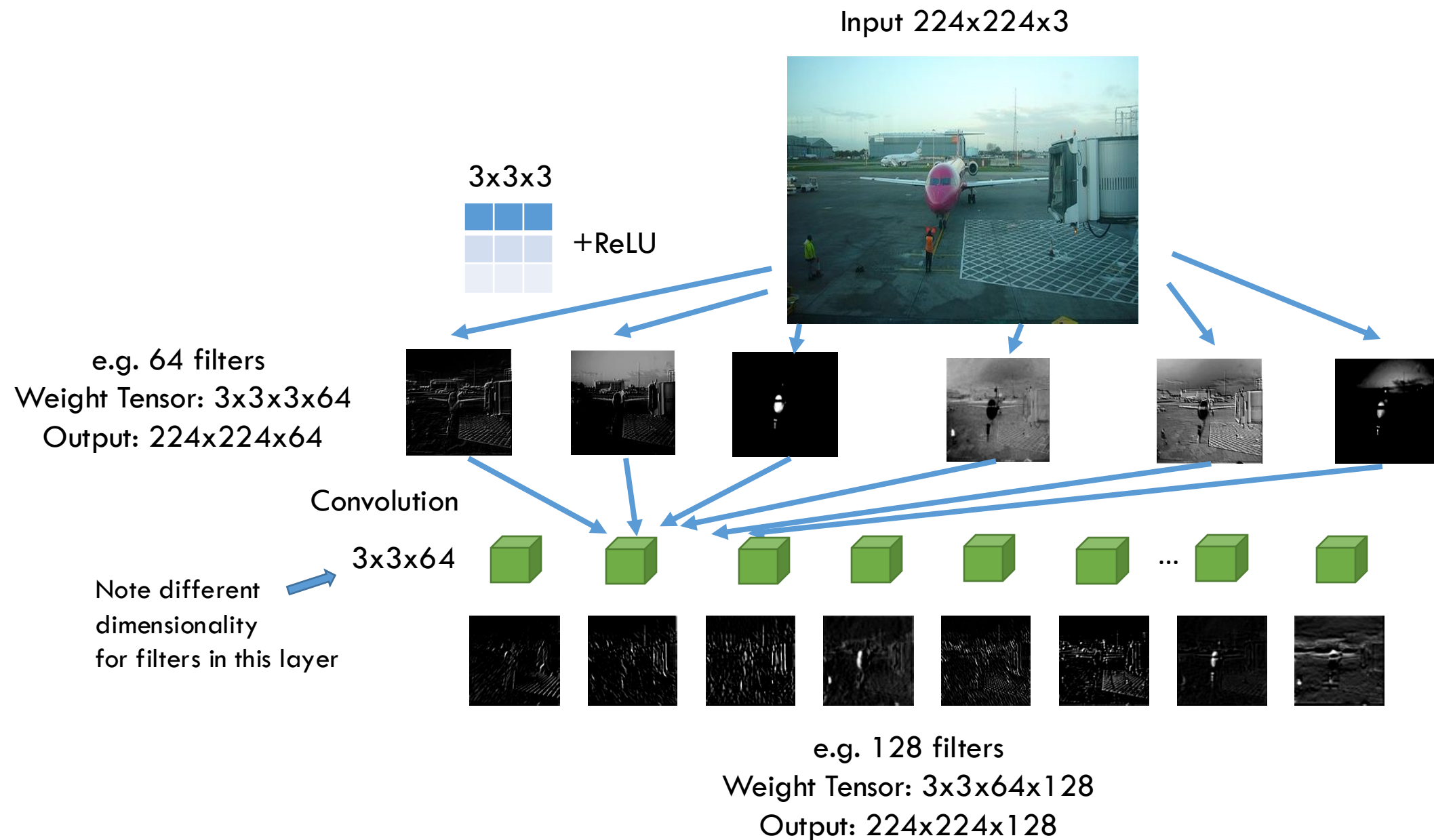
What happens after the first layer?

Simpler features are used to build more complicated features





2nd Layer in a CNN



Today's Learning Objectives

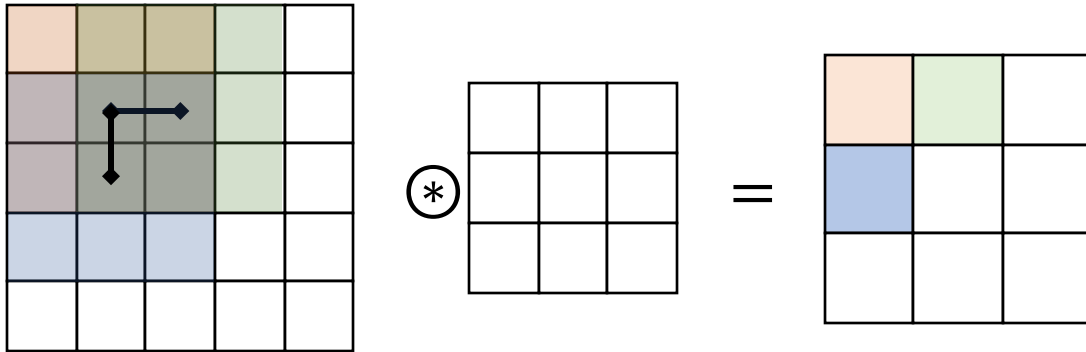
Be able to answer:

- ~~What are convolutional layers?~~
 - ~~What is convolution and what bias does it provide?~~
 - What is padding? Stride?
 - How are convolutional layers related to linear layers?
- What are pooling layers?
 - Why do we use them?
- What are convolutional neural networks?
 - What is a receptive field?





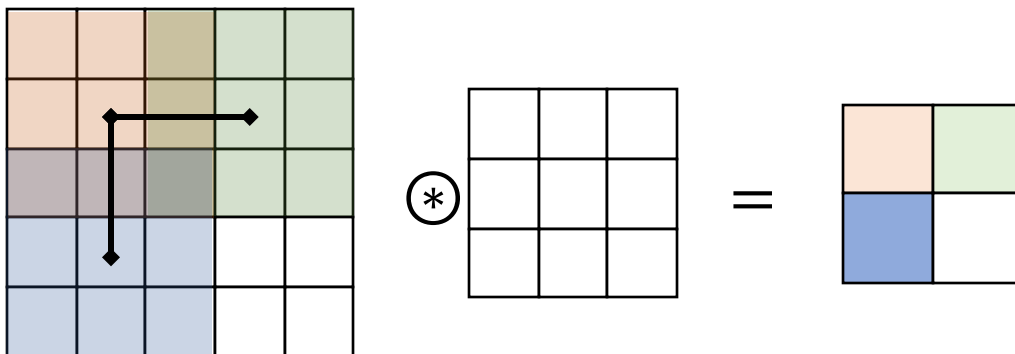
Stride 1



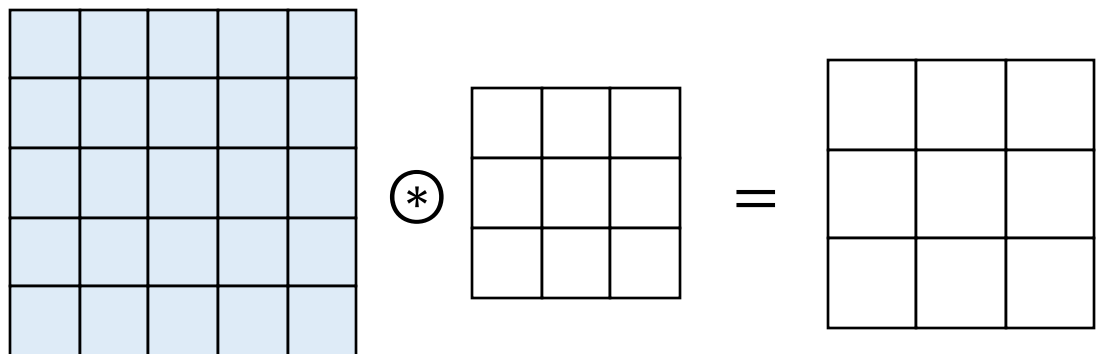
Stride of the convolution determines how to shift the filter after each computation

Larger stride → smaller output

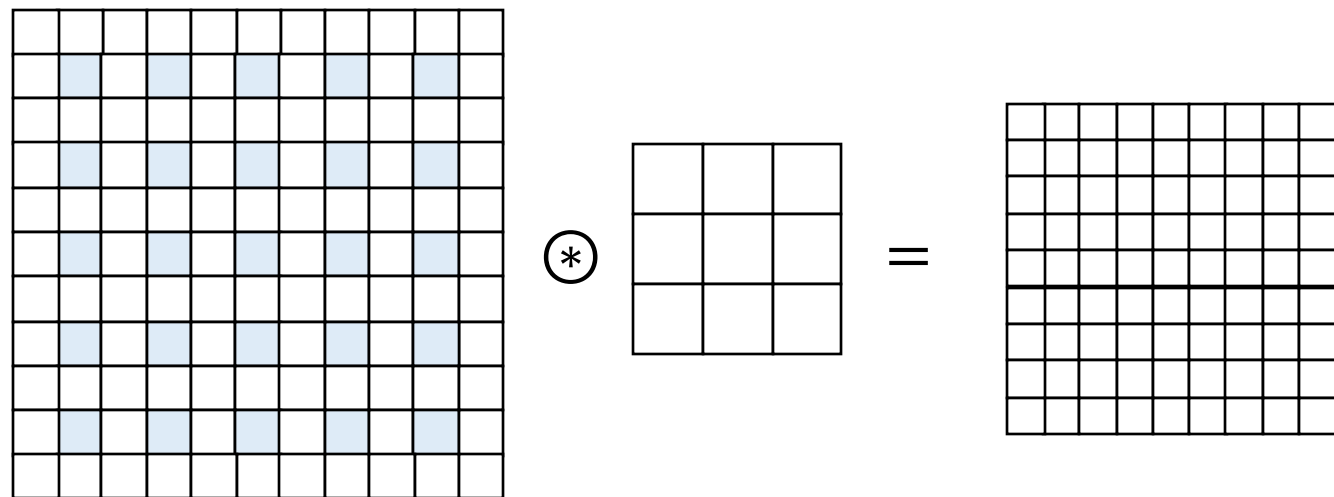
Stride 2



Stride 1



Stride 0.5 (requires $(k-1)/2$ padding)



Stride of the convolution determines how to shift the filter after each computation

Larger stride → smaller output

Fractional slides expand input with zeros, then convolve. Makes larger output than input.



Padding in general with padding of p and stride of s :

$$\begin{array}{c} w + 2p \\ \hline \begin{array}{c} h + 2p \\ \hline \end{array} \end{array} \circledast \begin{array}{c} k \\ \hline k \end{array} = \begin{array}{c} \frac{w + 2p - k}{s} + 1 \\ \hline \end{array} \begin{array}{c} \frac{h + 2p - k}{s} + 1 \\ \hline \end{array}$$

Today's Learning Objectives

Be able to answer:

- ~~What are convolutional layers?~~
 - ~~What is convolution and what bias does it provide?~~
 - ~~What is padding? Stride?~~
 - How are convolutional layers related to linear layers?
- What are pooling layers?
 - Why do we use them?
- What are convolutional neural networks?
 - What is a receptive field?





Contrasting Fully-Connected vs. Convolution

$$\begin{array}{|c|c|} \hline Z \\ \hline z_{11} & z_{12} \\ \hline z_{21} & z_{22} \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline X \\ \hline x_{11} & x_{12} & x_{13} \\ \hline x_{21} & x_{22} & x_{23} \\ \hline x_{31} & x_{32} & x_{33} \\ \hline \end{array} \circledast \begin{array}{|c|c|} \hline W \\ \hline w_{11} & w_{12} \\ \hline w_{21} & w_{22} \\ \hline \end{array}$$

$$z_{11} = w_{11}x_{11} + w_{12}x_{12} + w_{21}x_{21} + w_{22}x_{22}$$

$$z_{12} = w_{11}x_{12} + w_{12}x_{13} + w_{21}x_{22} + w_{22}x_{23}$$

$$z_{21} = w_{11}x_{21} + w_{12}x_{22} + w_{21}x_{31} + w_{22}x_{32}$$

$$z_{22} = w_{11}x_{22} + w_{12}x_{23} + w_{21}x_{32} + w_{22}x_{33}$$

Consider a fully-connected layer $\mathbf{z} = \mathbf{x}W$ where the input $\mathbf{x}$ and output $\mathbf{z}$ are vectorized versions of X and Z , we could learn the same mapping as the convolution by setting W

$$\begin{bmatrix} z_{11} & z_{12} & z_{21} & z_{22} \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{21} & x_{22} & x_{23} & x_{31} & x_{32} & x_{33} \end{bmatrix} \begin{bmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ 0 & w_{12} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ 0 & w_{22} & 0 & w_{12} \\ 0 & 0 & w_{21} & 0 \\ 0 & 0 & w_{22} & w_{21} \\ 0 & 0 & 0 & w_{22} \end{bmatrix}$$



Convolutional mapping:

$$[z_{11} \ z_{12} \ z_{21} \ z_{22}] = [x_{11} \ x_{12} \ x_{13} \ x_{21} \ x_{22} \ x_{23} \ x_{31} \ x_{32} \ x_{33}] \begin{bmatrix} w_{11} & 0 & 0 & 0 \\ w_{12} & w_{11} & 0 & 0 \\ 0 & w_{12} & 0 & 0 \\ w_{21} & 0 & w_{11} & 0 \\ w_{22} & w_{21} & w_{12} & w_{11} \\ 0 & w_{22} & 0 & w_{12} \\ 0 & 0 & w_{21} & 0 \\ 0 & 0 & w_{22} & w_{21} \\ 0 & 0 & 0 & w_{22} \end{bmatrix}$$

Fully-connected mapping:

$$[z_{11} \ z_{12} \ z_{21} \ z_{22}] = [x_{11} \ x_{12} \ x_{13} \ x_{21} \ x_{22} \ x_{23} \ x_{31} \ x_{32} \ x_{33}] \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \\ w_{51} & w_{52} & w_{53} & w_{54} \\ w_{61} & w_{62} & w_{63} & w_{64} \\ w_{71} & w_{72} & w_{73} & w_{74} \\ w_{81} & w_{82} & w_{83} & w_{84} \\ w_{91} & w_{92} & w_{93} & w_{94} \end{bmatrix}$$



Contrasting Fully-Connected vs. Convolution

Consider a fully-connected layer mapping a 256-by-256 image to 500 hidden units:

- Number of parameters: $(256*256*3)*500 = 98.3$ million

Consider a convolutional layer on a 256-by-256 image with 500 11x11 filters?

- Number of parameters: $(11*11*3)*500 = 181,500$

Consider 3 convolutional layers on a 256-by-256 image with 500 3x3 filters each:

- Number of parameters: $(3*3*3)*500 + (3*3*500)*500 + (3*3*500)*500 = 4.5$ million
 - Would need about 45 of these networks to match the parameter count of a single fully connected!

Today's Learning Objectives

Be able to answer:

- ~~What are convolutional layers?~~
 - ~~What is convolution and what bias does it provide?~~
 - ~~What is padding? Stride?~~
 - ~~How are convolutional layers related to linear layers?~~
- What are pooling layers?
 - Why do we use them?
- What are convolutional neural networks?
 - What is a receptive field?





- Keeping inputs at full spatial dimensions is expensive.
- Often input signals will have noise.
- Patterns will be slightly deformable.



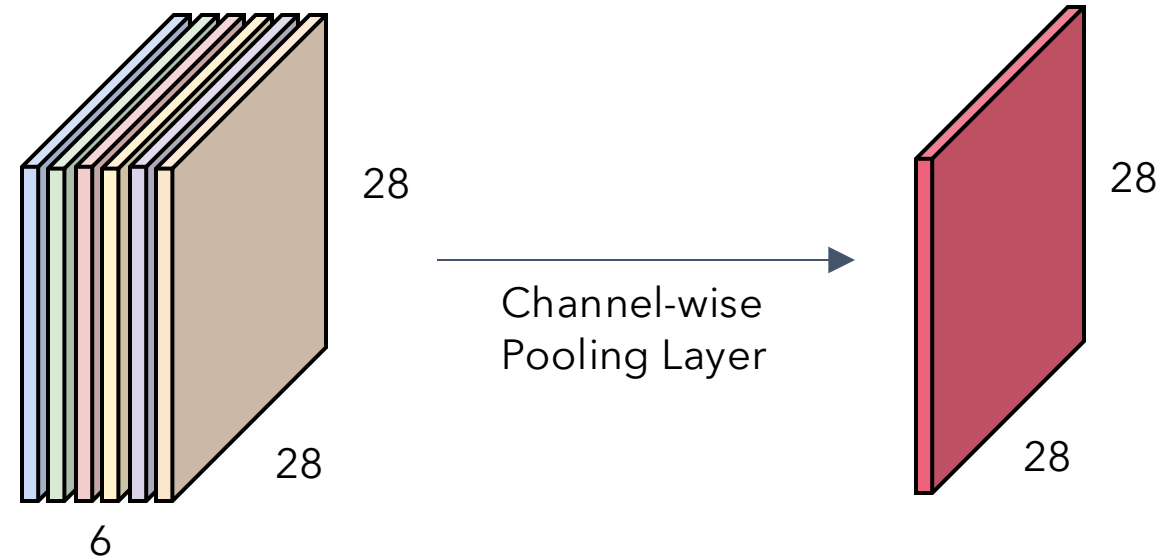
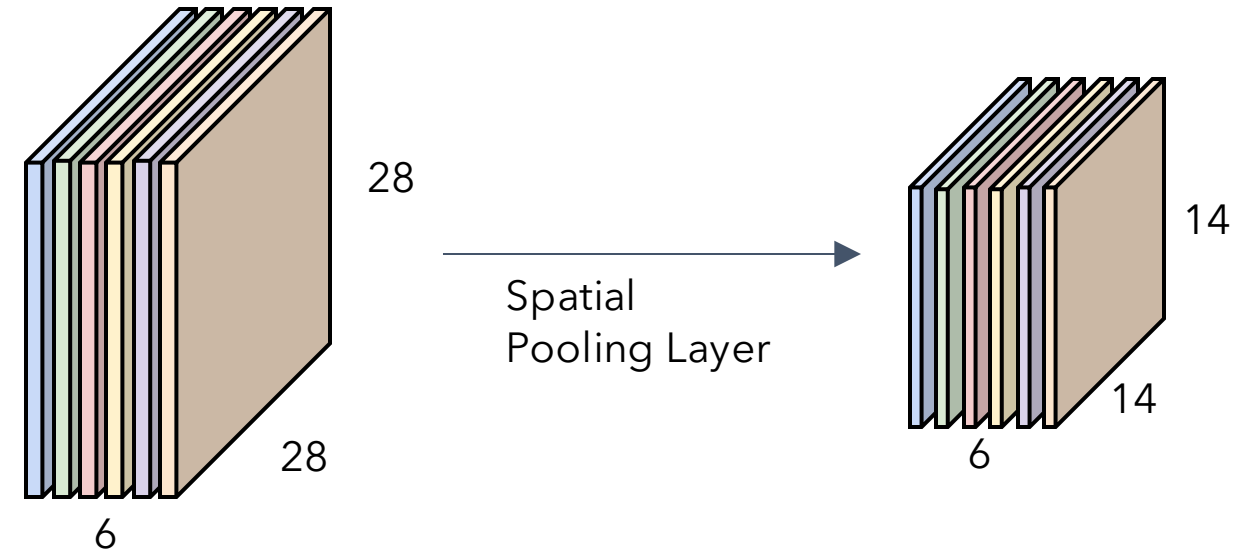
Idea: We can downsample and summarize local regions of an input.



Pooling Layers

Can perform parameter-free operations to reduce spatial or channel dimensions. Call these **pooling layers**.

- Spatial pooling
 - Replace a spatial area in each channel with a single value
- Channel pooling
 - Replace multiple channels at a location with a single value
- Common functions:
 - Max (max pooling)
 - Mean (mean pooling)





Two common patterns in spatial pooling:

Small kernel with stride equal to kernel size.

- Useful to subsample a spatial feature.
- Often found between blocks of padded convolution in modern networks.

Example of 2x2 mean pooling with stride 2:

6	2	-3	4
2	5	-7	2
0	3	5	6
-9	-8	2	0



3.75	-1
-3.5	3.25

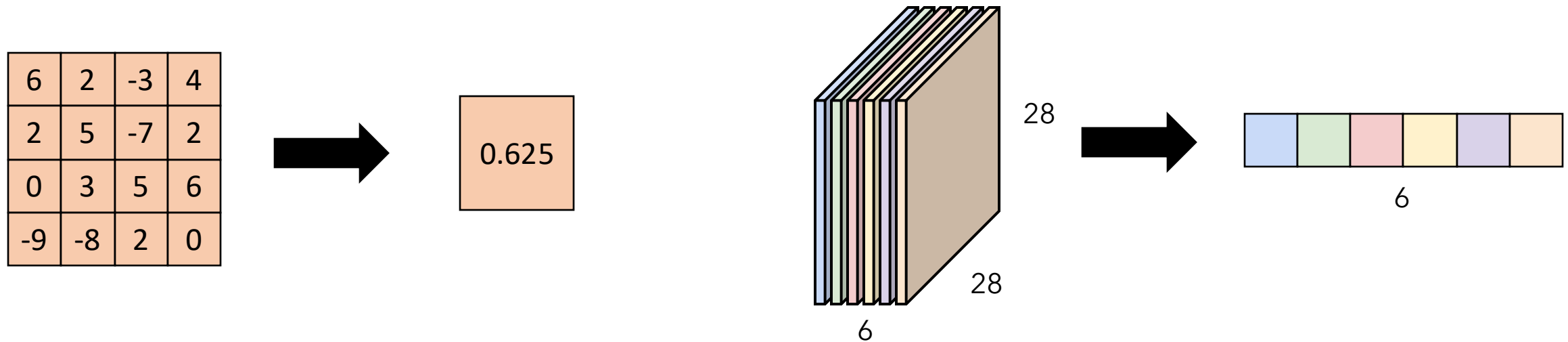


Two common patterns in spatial pooling:

Kernel the same size as the input feature

- Useful to accumulate information across all spatial dimensions.
- Often found at the end of image classification networks

Examples of a **global average pooling** below:

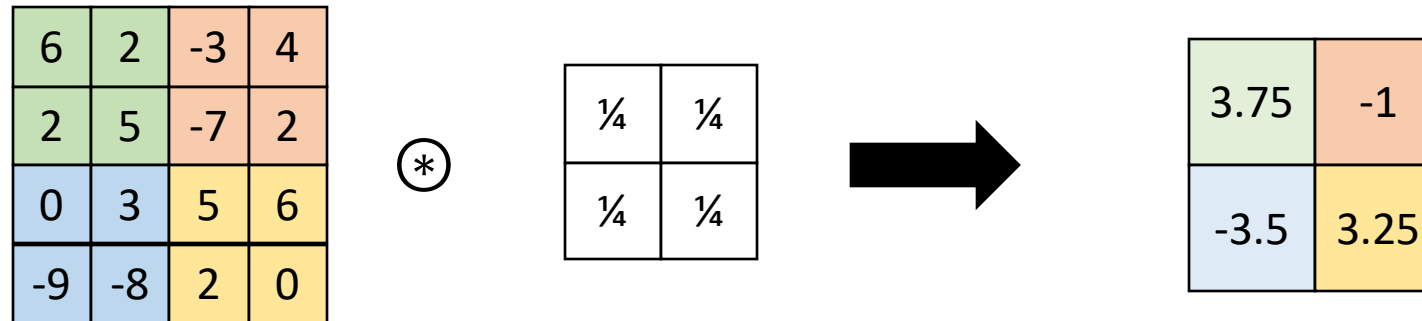




Implementing Mean Pooling as Convolution

Mean pooling can be computed as a convolution:

2x2 mean pooling with stride 2 as a convolution:



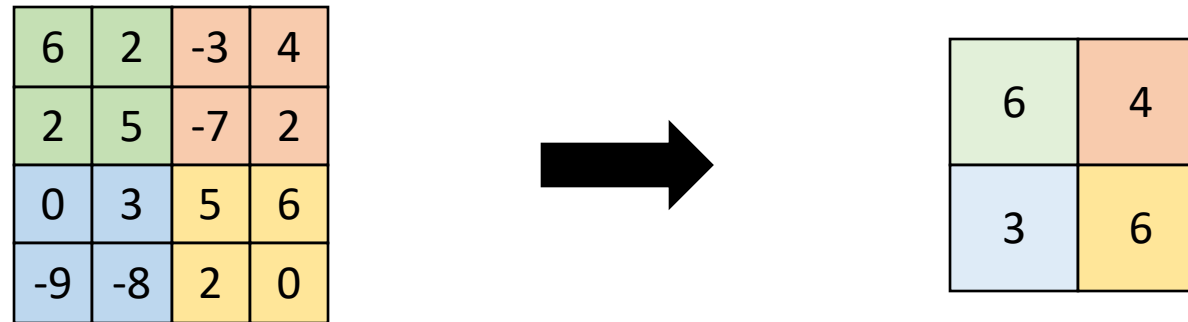
In general, $k \times k$ mean pooling with stride k as a convolution uses a $k \times k$ filter with entries $1/k$.

- Forward and backward passes same as convolution.
- Note, this would be done separately for each channel.

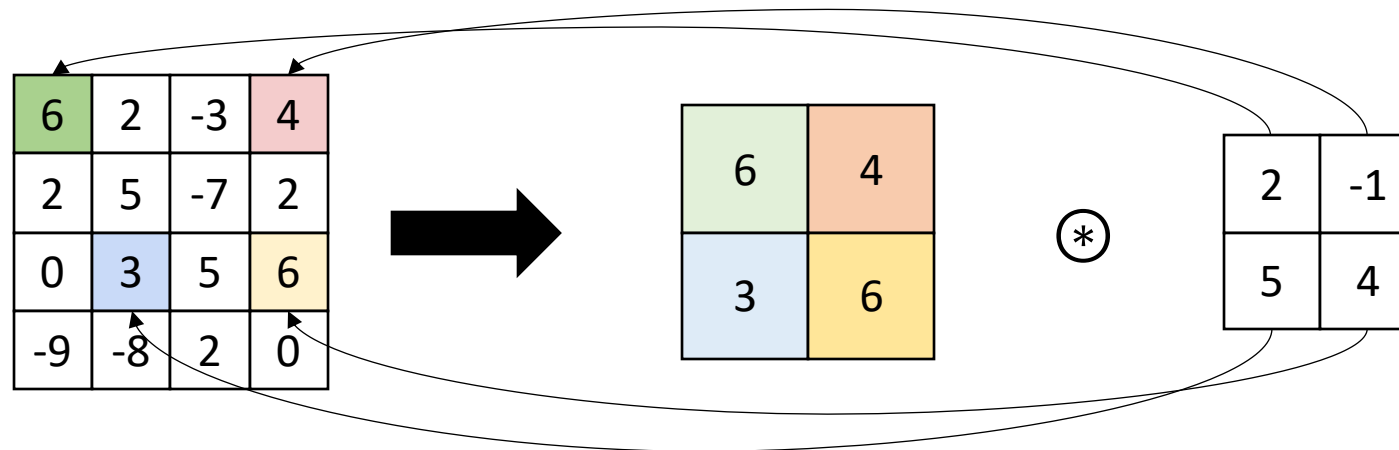


Max Pooling

Example 2x2 max pooling with stride 2. Can think of “get strongest response in feature map”.



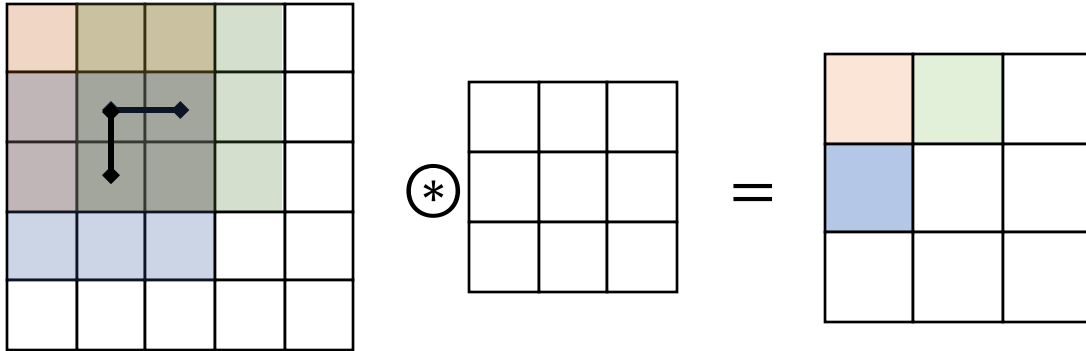
Allows some deformation:



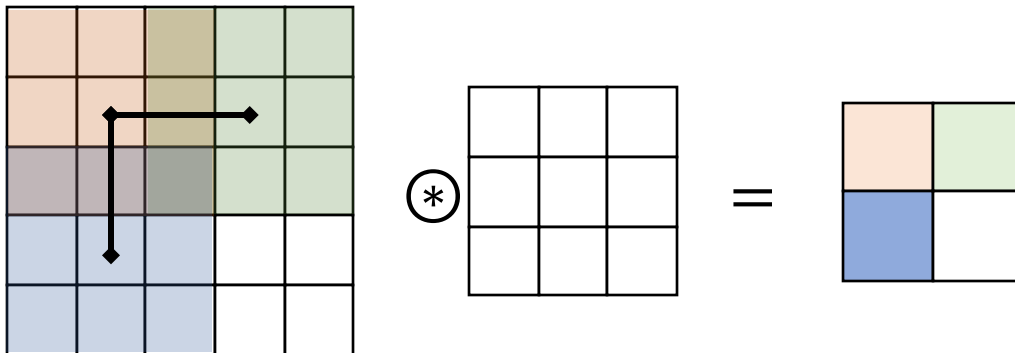


Recall: Convolutional Stride

Stride 1



Stride 2



Stride of the convolution determines how to shift the filter after each computation

Larger stride → smaller output

Can use this for learnable downsampling as well.

Today's Learning Objectives

Be able to answer:

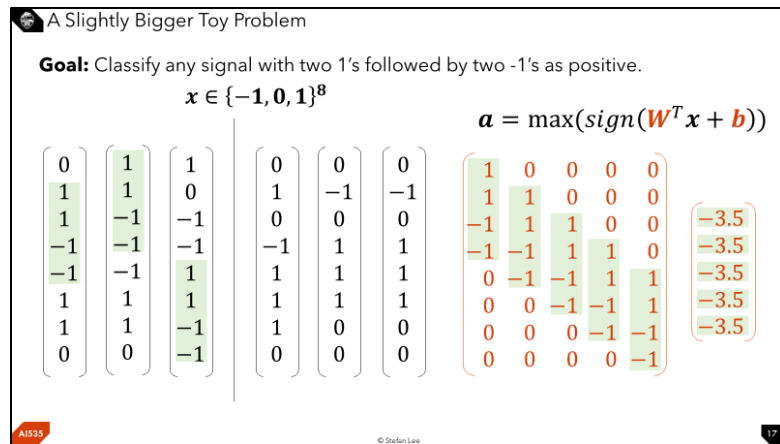
- What are convolutional layers?
 - What is convolution and what bias does it provide?
 - What is padding? Stride?
 - How are convolutional layers related to linear layers?
- ~~What are pooling layers?~~
 - ~~Why do we use them?~~
- What are convolutional neural networks?
 - What is a receptive field?





Where are we now?

Started at fully-connected neural networks, but noticed they are very parameter inefficient for cases where $f(x)$ is determined by presence/absence of local patterns.



- This is a pretty common feature of time or spatially varying signals like images or sound.
 - E.g. a picture of a cat relies on there being “cat features” somewhere in the image – doesn’t matter where.

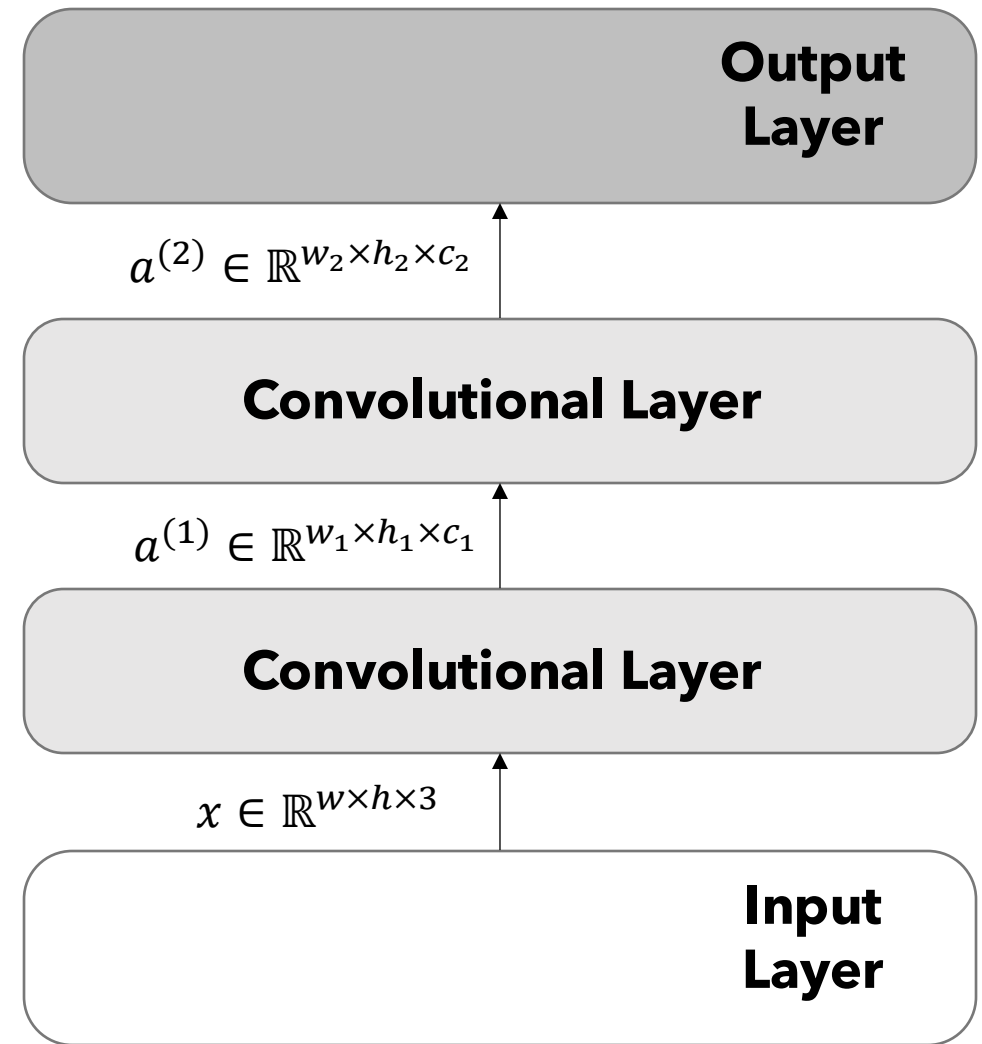
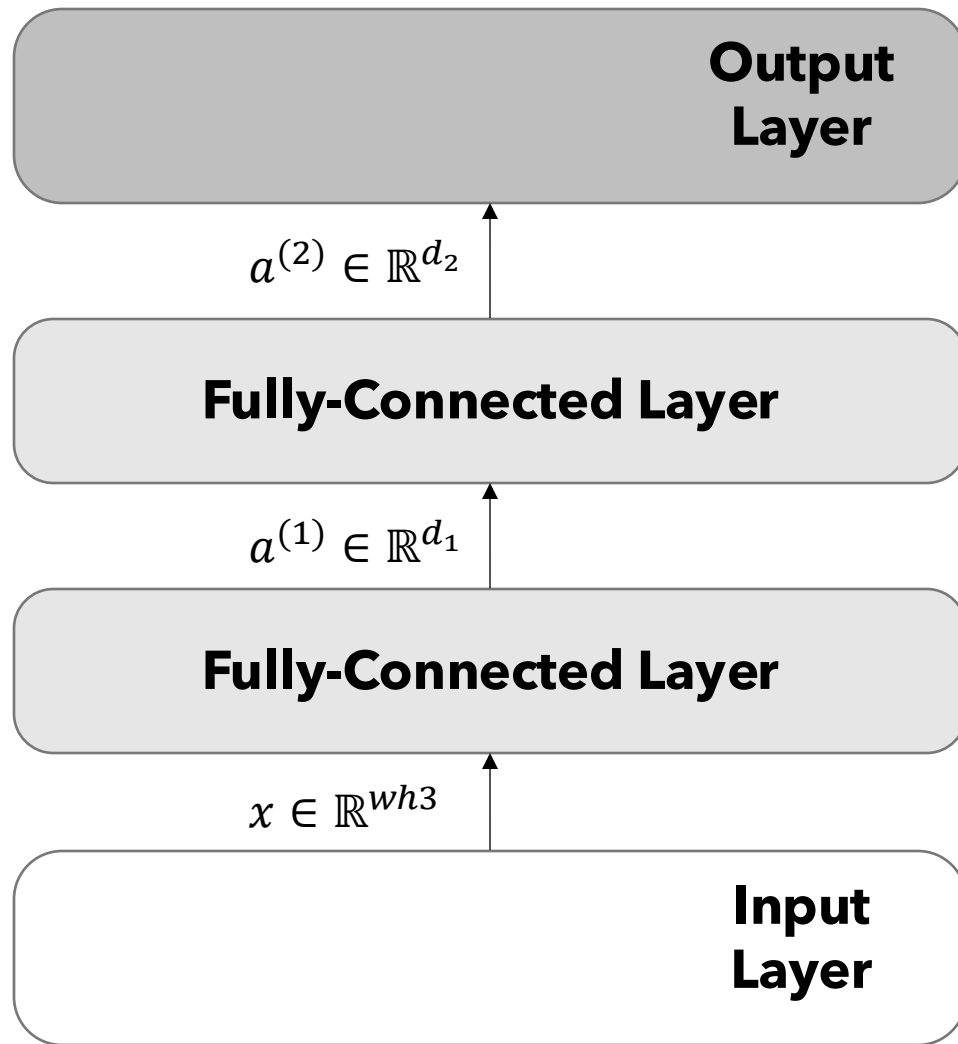
Added convolution as an operation that has this translational invariance property.

- Showed it is a linear operation with highly-sparse weight matrix and shared weights.

Added pooling layers to reduce spatial extent and apply a sort of “low-pass filter”



Fully-Connected vs Convolutional Networks

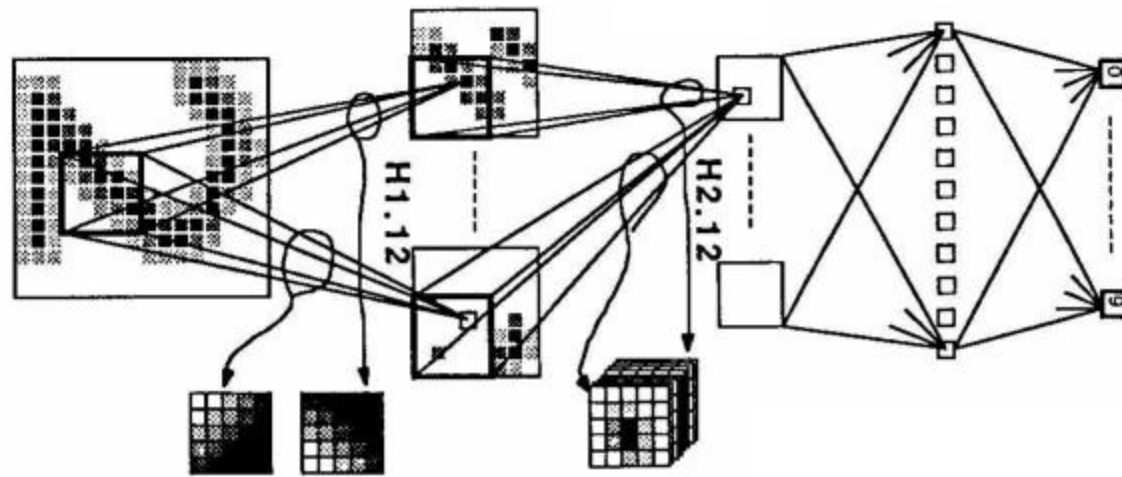




History: First Convolutional Neural Networks: LeNet

Introduced by Yann LeCunn et al., 1989.

- Task to recognize handwritten digits
- Sequence of convolutional layers with non-linear activations
- Around 10,000 parameters



3.3 Network Architecture. The network is represented in Figure 2. Its architecture is a direct extension of the one proposed in LeCun (1989). The network has three hidden layers named H1, H2, and H3, respectively. Connections entering H1 and H2 are local and are heavily constrained.

H1 is composed of 12 groups of 64 units arranged as 12 independent 8 by 8 feature maps. These 12 feature maps will be designated by H1.1, H1.2, ..., H1.12. Each unit in a feature map takes input on a 5 by 5 neighborhood on the input plane. For units in layer H1 that are one unit apart, their receptive fields (in the input layer) are two pixels apart. Thus, the input image is *undersampled* and some position information is eliminated. A similar two-to-one undersampling occurs going from layer H1 to H2. The motivation is that high resolution may be needed to detect the presence of a feature, while its exact position need not be determined with equally high precision.

It is also known that the kinds of features that are important at one place in the image are likely to be important in other places. Therefore, corresponding connections on each unit in a given feature map are constrained to have the same weights. In other words, each of the 64 units in H1.1 uses the same set of 25 weights. Each unit performs the same operation on corresponding parts of the image. The function performed by a feature map can thus be interpreted as a nonlinear subsampled convolution with a 5 by 5 kernel.

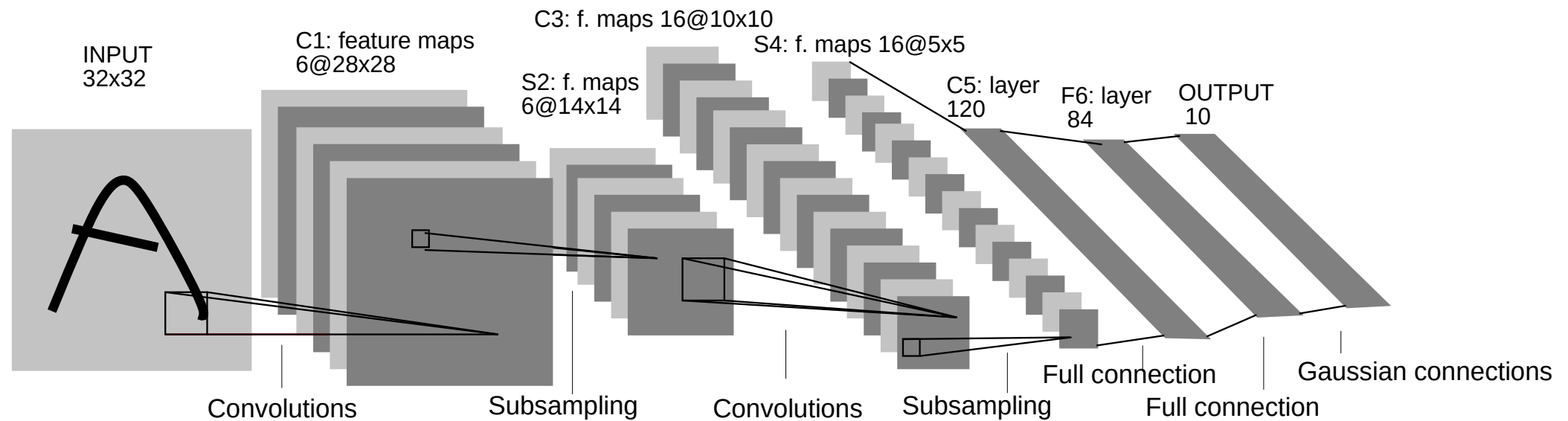
Of course, units in another map (say H1.4) share *another* set of 25 weights. Units do not share their biases (thresholds). Each unit thus has 25 input lines plus a bias. Connections extending past the boundaries of the input plane take their input from a virtual background plane whose



History: First Convolutional Neural Networks: LeNet-5

Increasingly modern version a decade later (Yann LeCun et al., 1999.)

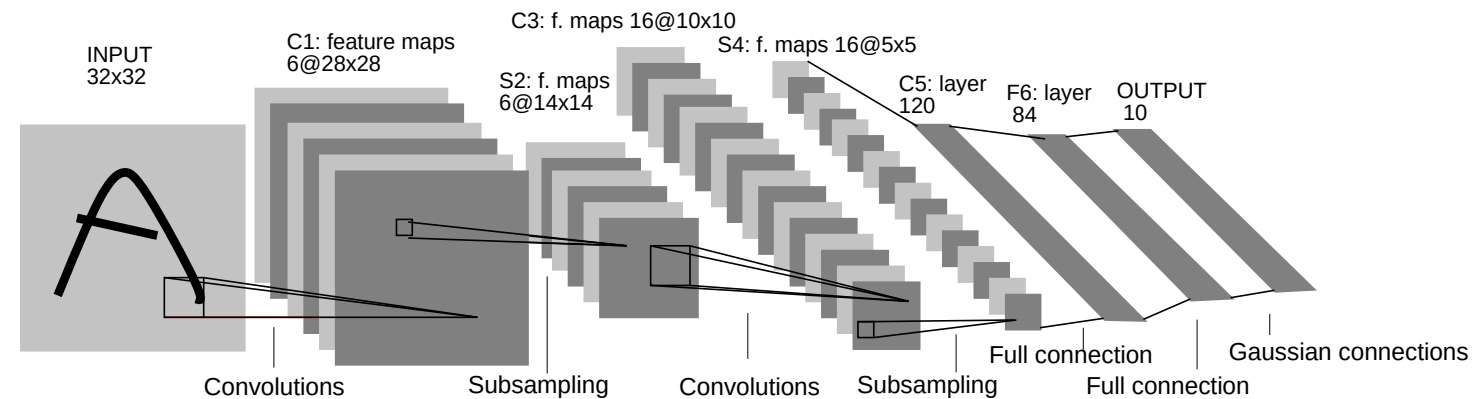
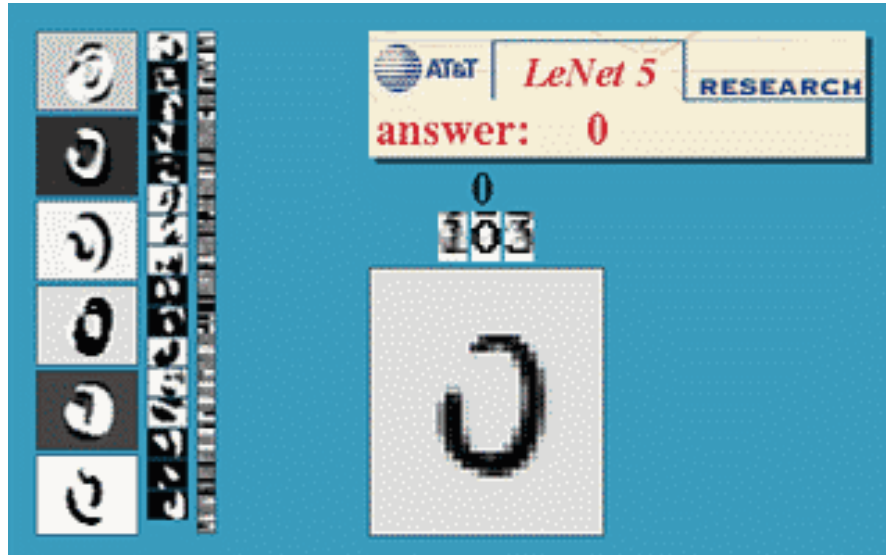
- Task to recognize handwritten digits for zipcodes
- Has many recognizable features present in modern networks still:
 - Many hidden layers
 - Many filters learned at each layer
 - Spatial resolution reduced via subsampling
- Around 60,000 parameters





History: First Convolutional Neural Networks: LeNet-5

Recognizing writing on ~10% of checks in North America.





History: First Convolutional Neural Networks: LeNet-5

Evaluating LeNet-5 on MNIST dataset of handwritten digits. Made 82 errors:





Computer vision community really flocked to CNNs after AlexNet's 2012 ImageNet result:

	LeNet (1989)	LeNet (1998)	AlexNet (2012)
classification task	digits	digits	objects
categories	10	10	1,000
image size	16×16	28×28	$256 \times 256 \times 3$
training examples	7,291	60,000	1.2 million
units	1,256	8,084	658,000
parameters	9,760	60,000	60 million
connections	65,000	344,000	652 million
total operations	11 billion	412 billion	200 quadrillion (est.)

Computers being faster / GPU-enabled really allows this idea of CNNs to shine.



Early Modern Convolutional Network: VGG16

arXiv:1409.1556v6 [cs.CV] 10 Apr 2015

Published as a conference paper at ICLR 2015

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* & Andrew Zisserman*

Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen, az}@robots.ox.ac.uk

ABSTRACT

In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16–19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the localisation and classification tracks respectively. We also show that our representations generalise well to other datasets, where they achieve state-of-the-art results. We have made our two best-performing ConvNet models publicly available to facilitate further research on the use of deep visual representations in computer vision.

1 INTRODUCTION

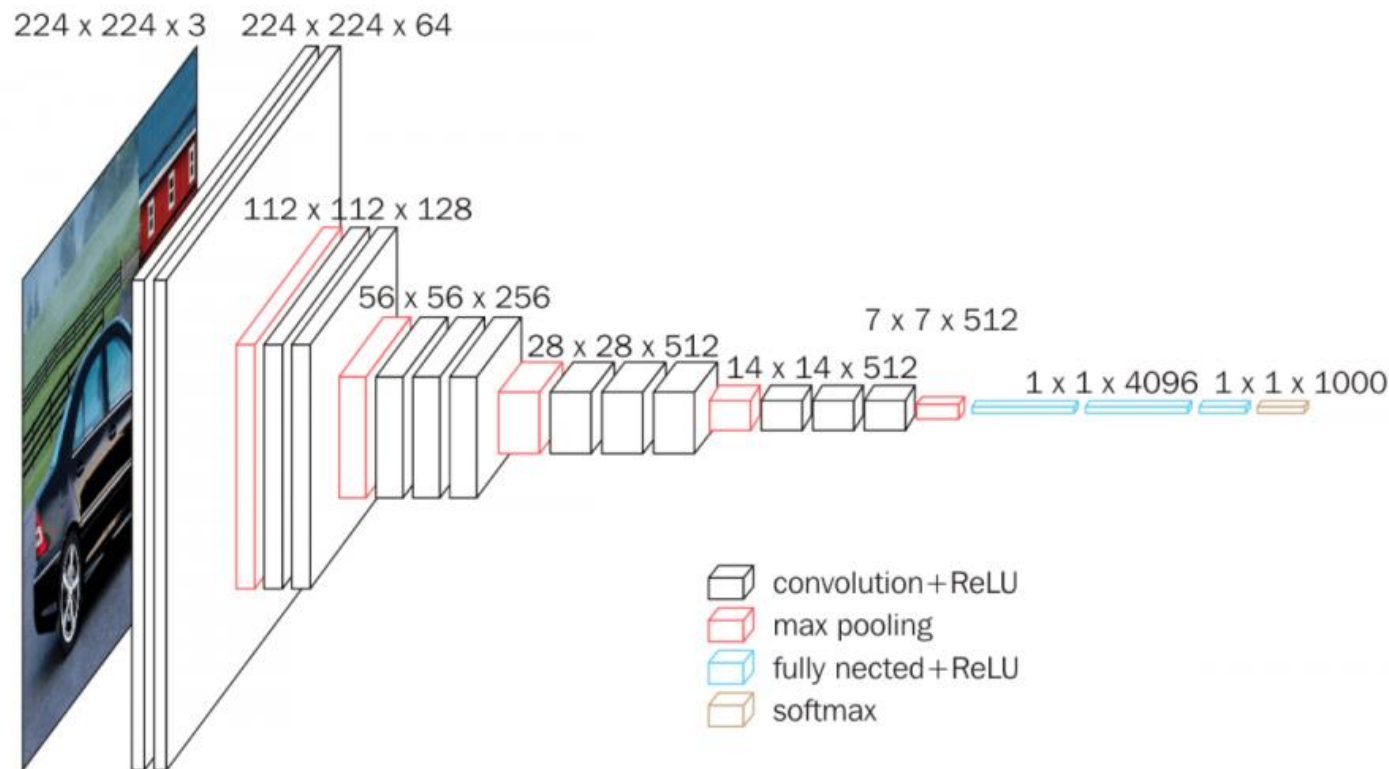
Convolutional networks (ConvNets) have recently enjoyed a great success in large-scale image and video recognition (Krizhevsky et al., 2012; Zeiler & Fergus, 2013; Sermanet et al., 2014; Simonyan & Zisserman, 2014) which has become possible due to the large public image repositories, such as ImageNet (Deng et al., 2009), and high-performance computing systems, such as GPUs or large-scale distributed clusters (Dean et al., 2012). In particular, an important role in the advance of deep visual recognition architectures has been played by the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) (Russakovsky et al., 2014), which has served as a testbed for a few generations of large-scale image classification systems, from high-dimensional shallow feature encodings (Perotini et al., 2010) (the winner of ILSVRC-2011) to deep ConvNets (Krizhevsky et al., 2012) (the winner of ILSVRC-2012).

With ConvNets becoming more of a commodity in the computer vision field, a number of attempts have been made to improve the original architecture of Krizhevsky et al. (2012) in a bid to achieve better accuracy. For instance, the best-performing submissions to the ILSVRC-2013 (Zeiler & Fergus, 2013; Sermanet et al., 2014) utilised smaller receptive window size and smaller stride of the first convolutional layer. Another line of improvements dealt with training and testing the networks densely over the whole image and over multiple scales (Sermanet et al., 2014; Howard, 2014). In this paper, we address another important aspect of ConvNet architecture design – its depth. To this end, we fix other parameters of the architecture, and steadily increase the depth of the network by adding more convolutional layers, which is feasible due to the use of very small (3×3) convolution filters in all layers.

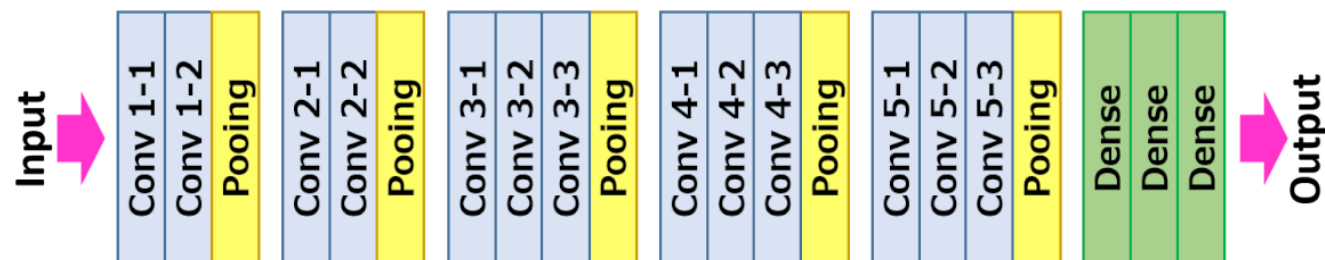
As a result, we come up with significantly more accurate ConvNet architectures, which not only achieve the state-of-the-art accuracy on ILSVRC classification and localisation tasks, but are also applicable to other image recognition datasets, where they achieve excellent performance even when used as a part of a relatively simple pipelines (e.g. deep features classified by a linear SVM without fine-tuning). We have released our two best-performing models¹ to facilitate further research.

The rest of the paper is organised as follows. In Sect. 2, we describe our ConvNet configurations. The details of the image classification training and evaluation are then presented in Sect. 3, and the

¹current affiliation: Google DeepMind ²current affiliation: University of Oxford and Google DeepMind
¹http://www.robots.ox.ac.uk/~vgg/research/very_deep/



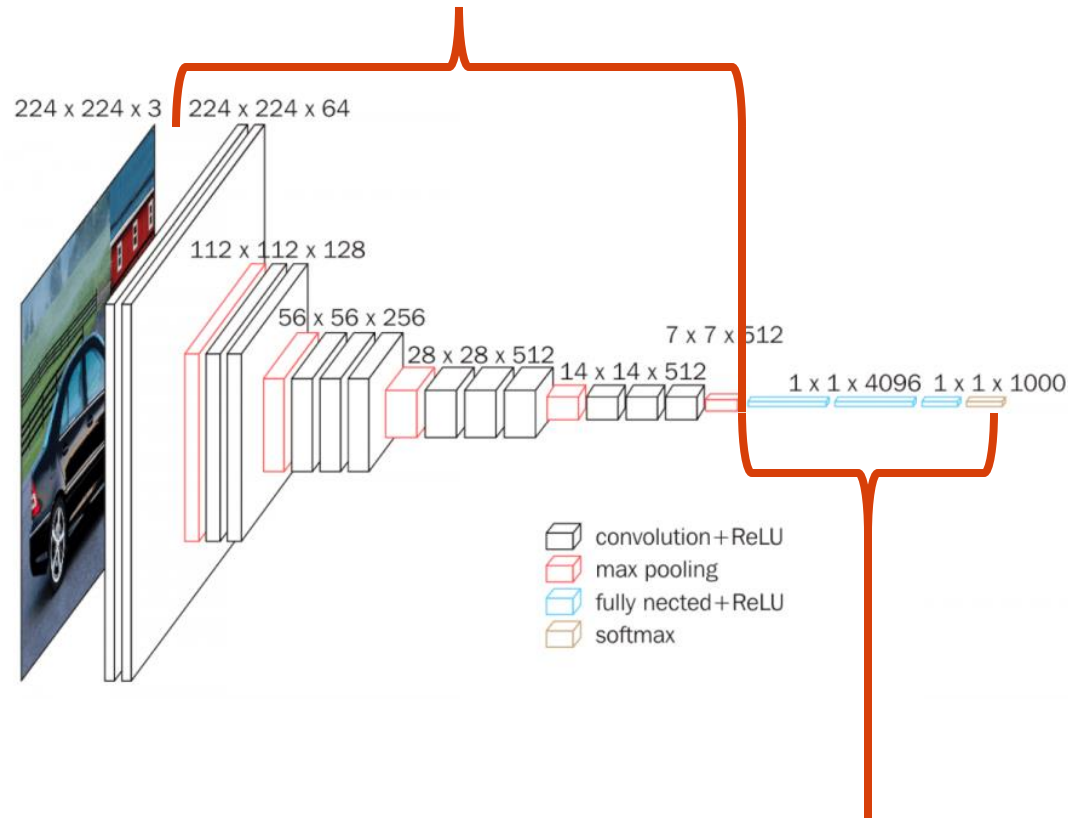
VGG-16





Early Modern Convolutional Network: VGG16

Sequence of convolutional layers with ReLU activations. Periodically reducing spatial resolution with max-pool.



Eventually vectorize the spatial map and apply a fully connected network that ends with a softmax output.

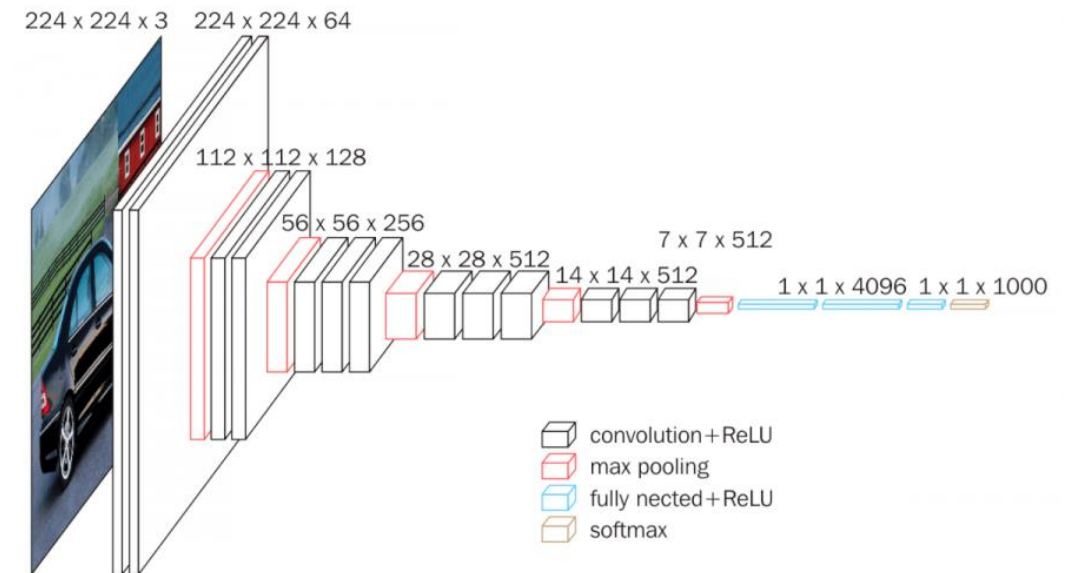
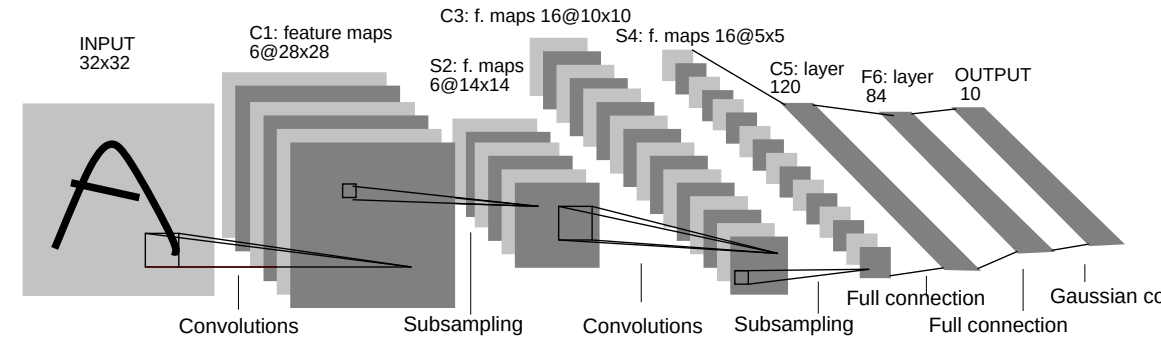


Early Modern Convolutional Network: VGG16

What is different from LeNet-5?

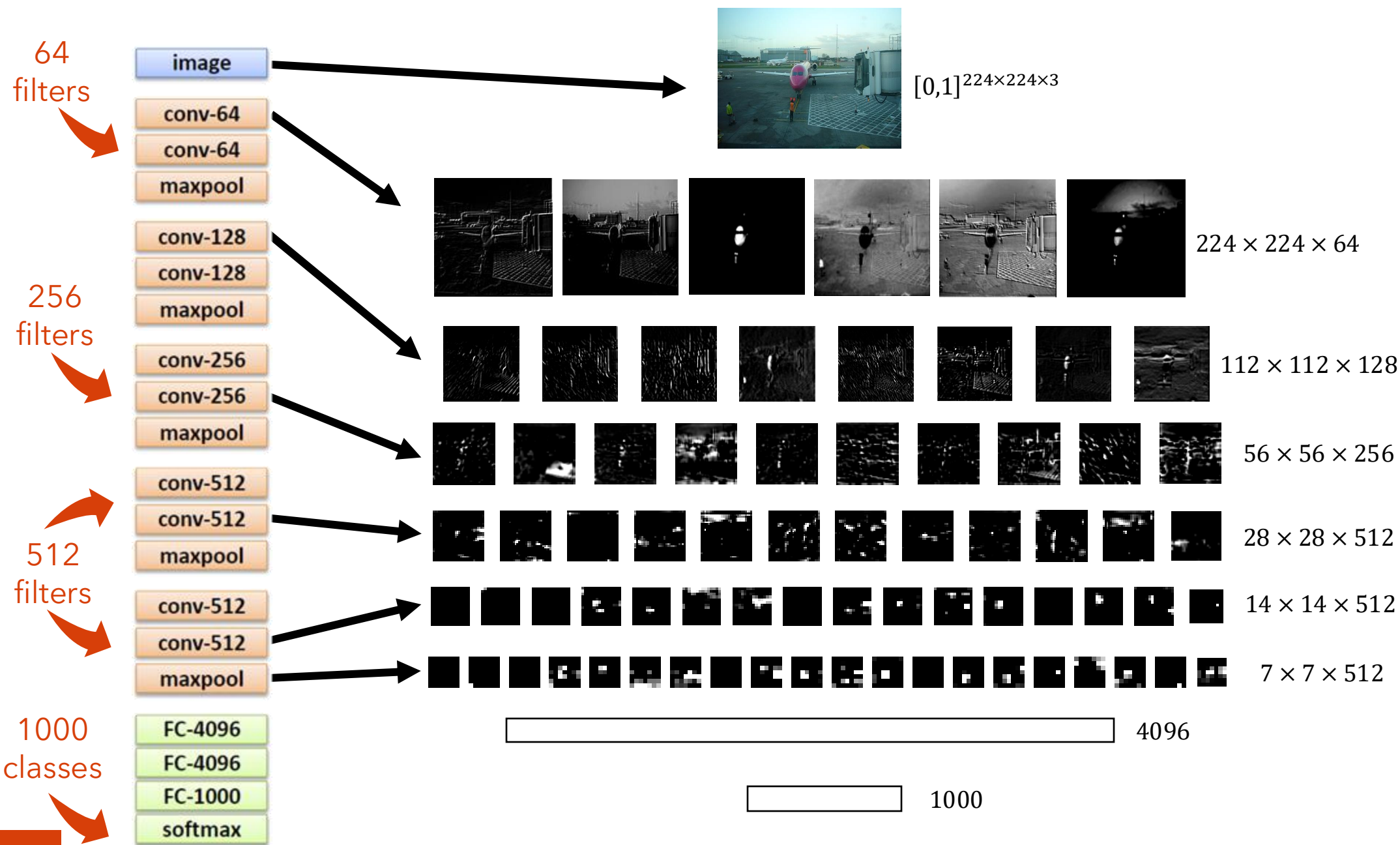
Modern convolutional neural networks tend towards:

- ReLU (or related) activation functions
- Small filter sizes (3x3 and less)
 - Typically zero-padded to not change spatial dimensions
- Small pooling sizes (e.g. 2x2)
 - Used periodically to reduce dim
- Small strides (typically 1)
- Many, many convolutional layers (aka “deep” networks) followed by fully-connected to perform the classification





Early Modern Convolutional Network: VGG16



Today's Learning Objectives

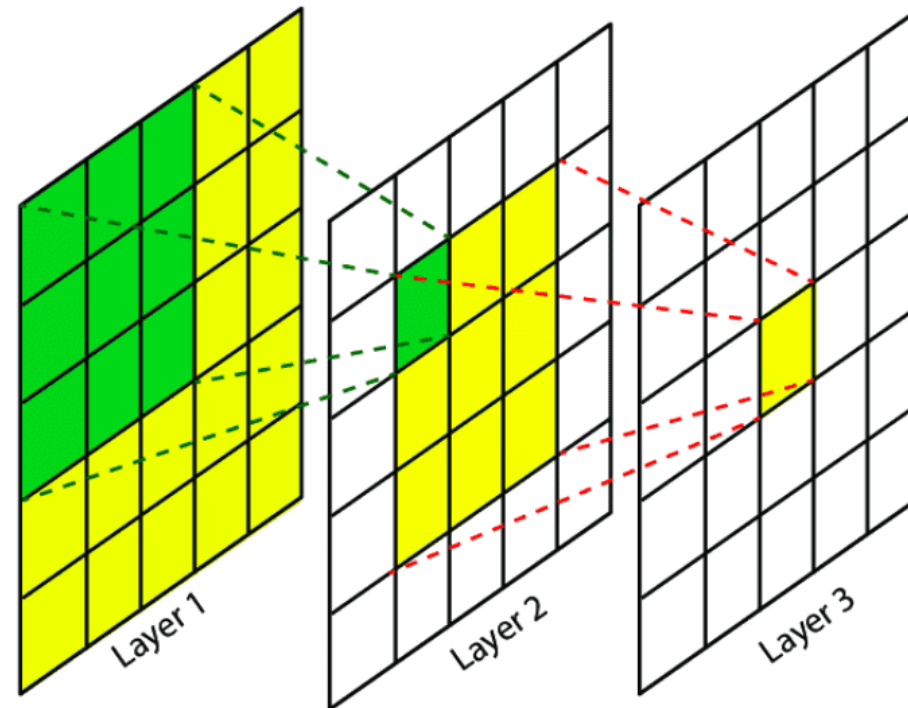
Be able to answer:

- What are convolutional layers?
 - What is convolution and what bias does it provide?
 - What is padding? Stride?
 - How are convolutional layers related to linear layers?
- What are pooling layers?
 - Why do we use them?
- What are convolutional neural networks?
 - What is a receptive field?





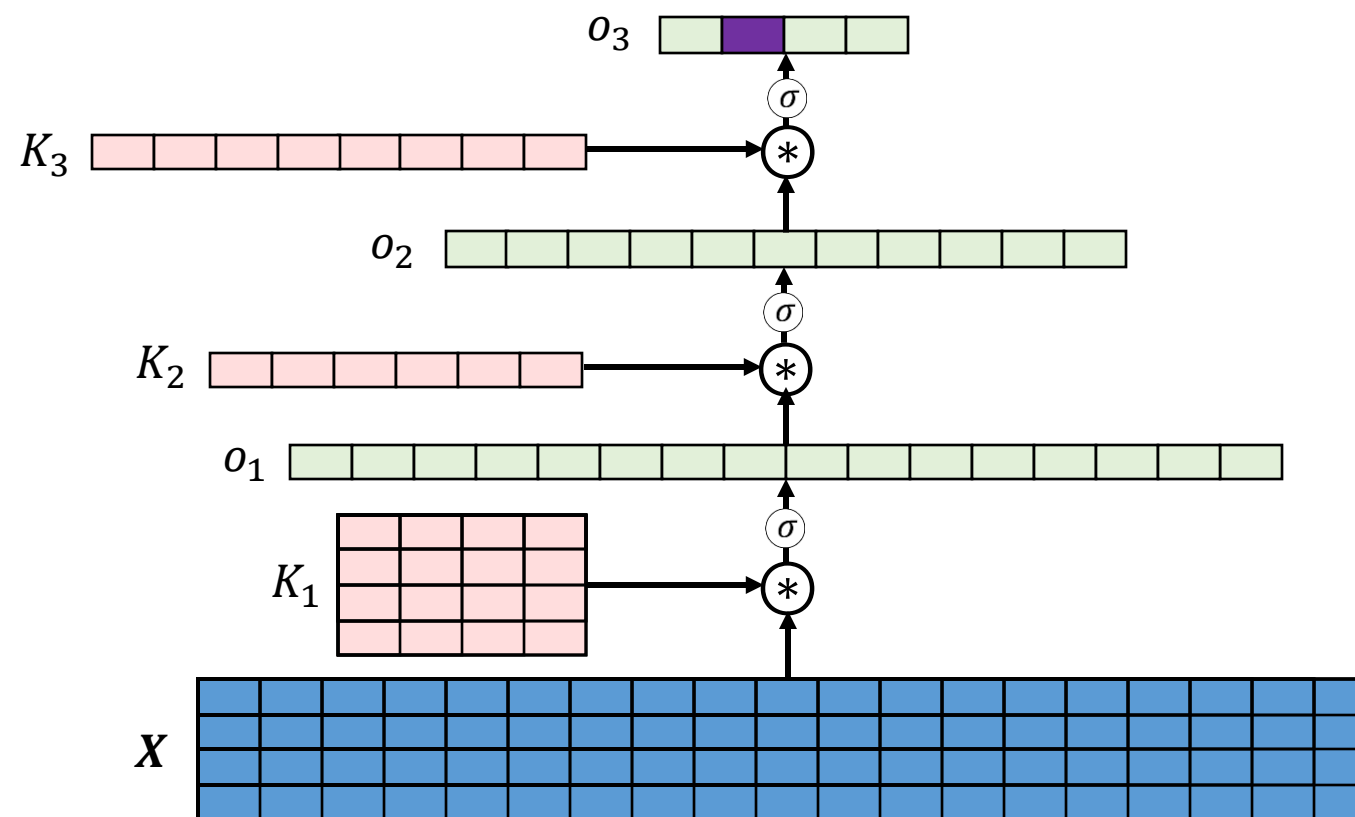
Small filter sizes (e.g., 3×3) can end up considering large portions of the image when stacked.



Call the region of the input “visible” to a single cell in an output the “receptive field”.

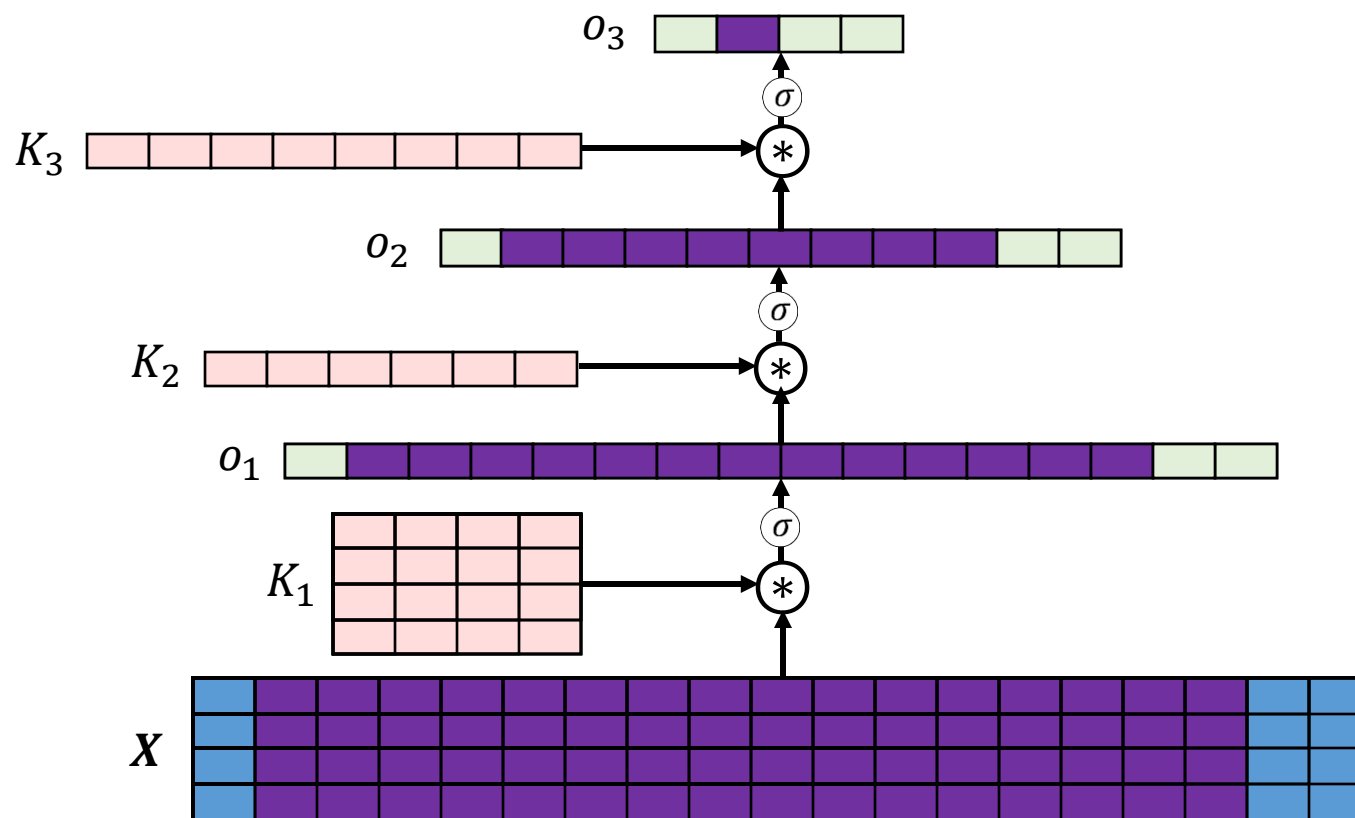


Consider the following stack of 1d convolutions and non-linearities. What elements of X can contribute to the element of o_3 marked in purple?





Consider the following stack of 1d convolutions and non-linearities. What elements of X can contribute to the element of o_3 marked in purple?



Today's Learning Objectives

Be able to answer:

- What are convolutional layers?
 - What is convolution and what bias does it provide?
 - What is padding? Stride?
 - How are convolutional layers related to linear layers?
- What are pooling layers?
 - Why do we use them?
- What are convolutional neural networks?
 - What is a receptive field?

